

Low Communication Threshold FHE from Standard (Module-)LWE

Hiroki Okada^{1,2} and Tsuyoshi Takagi²

¹ KDDI Research, Inc., Saitama, Japan

² The University of Tokyo, Tokyo, Japan

Abstract. Threshold fully homomorphic encryption (ThFHE) is a multi-party extension of FHE; any subset of at least T out of N parties can decrypt the ciphertexts by combining their *decryption shares*. Recently, Passelègue and Stehlé (Asiacrypt 2024) presented a ThFHE scheme with polynomially short decryption shares from the “known-norm” variant of learning with errors (LWE) assumption, in which the norm of the secret key is leaked to the adversary. While known-norm LWE is reduced from standard LWE, its module extension, known-covariance module-LWE (MLWE), lacks a known reduction from standard MLWE. Hence, extending their ThFHE scheme to the MLWE-based construction remains an open question.

In this paper, we address this open problem: We construct a ThFHE scheme with polynomially small decryption shares from standard LWE/MLWE. Our core technique, which we call *noise padding*, eliminates the need of known-norm variants of LWE. We distribute shares of a *padding noise* and use them to adjust the distribution of decryption noise so that no information about the secret key is leaked. Furthermore, our ThFHE efficiently realizes arbitrary T -out-of- N threshold decryption via simple Shamir secret sharing instead of $\{0, 1\}$ -linear secret sharing. Hence, the sizes of the keys, ciphertexts and decryption shares in our scheme are compact: they are $O(1)$ w.r.t. the number of parties N .

1 Introduction

Fully homomorphic encryption (FHE) was first realized by Gentry [Gen09]. Constructions based on the learning with errors (LWE) problem [Reg09] and its variants ring-LWE (RLWE) [LPR10] or module-LWE (MLWE) [BGV12] are efficient, leading to active research in this field [CKKS17; CGGI20]. Threshold FHE (ThFHE) is an extension of FHE in which ciphertexts can be decrypted by collecting *partial decryption shares* from T out of N parties, where N is the number of parties and T ($\leq N$) is a threshold. In 2023, the US agency NIST

This preprint is the full version of the paper published in the proceedings of Asiacrypt 2025 [OT26]. The paper is available at: https://doi.org/10.1007/978-981-95-5122-4_6. See Sect. 1.3 for a summary of the changes.

initiated a project to establish guidelines and recommendations for implementing threshold cryptosystems, including ThFHE [BP23]. ThFHE is an important cryptographic tool that can be applied to construct round optimal MPC protocols [GGHR14; GLS15; BJMS20] and *universal thresholdizer* [BGG⁺18], which can be used to add threshold functionality to, e.g., CCA-secure PKE, signature schemes, pseudorandom functions (PRF) and functional encryptions.

ThFHE was first constructed by Asharov et al. [AJL⁺12], and Boneh et al. [BGG⁺18] proposed a ThFHE with arbitrary T -out-of- N decryption. A notable drawback of these schemes is that they require a super-polynomially large modulus, leading to super-polynomially long keys, ciphertexts and decryption shares. To address this drawback, Boudgoust and Scholl [BS23] proposed a ThFHE scheme with polynomially small modulus, providing game-based security. However, Passelègue and Stehlé [PS25] recently noted a flaw in the security proof of [BS23]. In response to this attack, Boudgoust and Scholl updated their scheme in [BS24], and their scheme now achieves only *selective* security, where the adversary is allowed to query the encryption (and homomorphic evaluation) oracle only before receiving challenge ciphertexts and partial decryption shares.

In addition to the cryptanalysis, Passelègue and Stehlé [PS25] proposed a simulation-secure ThFHE scheme with polynomially short decryption shares, realizing threshold decryption with a small communication size. Although this scheme still requires a super-polynomially large modulus for the input ciphertexts, the communication size of sending them can be reduced by *transcipherring* [NLV11; BCK⁺23] at the expense of homomorphic evaluation of the decryption circuit of symmetric key encryption (SKE) such as AES or FHE-friendly SKEs [ARS⁺15; DEG⁺18; HKL⁺22]. Hence, we can achieve threshold FHE with small input/output communications.

One notable drawback of the ThFHE scheme of [PS25] is that it cannot be extended to the construction from MLWE (or RLWE). Thus, it cannot be applied to most instantiations of FHE, including [CKKS17; CGGI20]. This is because the ThFHE of [PS25] relies on “known-norm” LWE (knLWE) [MS25] where the norm of the secret vector is given to the adversary. (Precisely, [PS25] uses “yet another” LWE (yaLWE), which is a generalization of known-norm LWE.) Since there exists a reduction from (standard) LWE to knLWE [MS25], ThFHE of [PS25] can be based on LWE. To extend their scheme into an MLWE-based construction, a “known-covariance” variant of MLWE is needed. However, no reduction from standard assumption such as MLWE to the known-covariance MLWE is known. Hence, Passelègue and Stehlé posed the following open question:

Question 1.1. Can we construct (simulation-secure) threshold FHE with polynomially short decryption shares from MLWE (or RLWE)?

Another drawback in the ThFHE scheme of [PS25] and prior works ([BS23] and [BGG⁺18, Constr. 5.6]) is that they are based on the $\{0, 1\}$ -linear secret sharing scheme ($\{0, 1\}$ -LSSS). The size of secret key shares and decryption shares $\{0, 1\}$ -LSSS are large for T -out-of- N threshold decryption: Each key

Table 1. Comparison of ThFHE schemes with (arbitrary) T -out-of- N decryption. Q and q represents the modulus of ciphertexts and decryption shares, respectively. For the rows of “Dec share size” and “sk share size”, we write the number of elements (over $\mathbb{Z}_q$ and $\mathbb{Z}_q^n$) included in decryption and secret shares, respectively. L_{Dec} represents the bound on the decryption query; we write $L_{\text{Dec}} = \infty$ if it is unbounded. The row “Sim. Sec.” marks schemes with simulation security. The row “Module-LWE” marks which schemes can be constructed from MLWE. The row “Without server” marks which schemes can be constructed without a semi-honest and uncorrupted server.

	[BGG ⁺ 18]	[BS23]	[PS25]	Ours	
				§5	§6, §7
Enc mod. (Q)	$\Omega(N^{4.2})\kappa^{\omega(1)}$	$\Omega(N^{4.2})\kappa^{O(1)}$	$\Omega(N^{4.2})\kappa^{\omega(1)\dagger}$	$\kappa^{\omega(1)\dagger}$	$\kappa^{\omega(1)\dagger}$
Dec mod. (q)	$\Omega(N^{4.2})\kappa^{\omega(1)}$	$\Omega(N^{4.2})\kappa^{O(1)}$	$\Omega(N^{4.2})\kappa^{O(1)}$	$\kappa^{O(1)}$	$\kappa^{O(1)}$
Dec share size	$O(N^{4.2})$	$O(N^{4.2})$	$O(N^{4.2})$	$O(1)$	$O(1)$
sk share size	$O(N^{4.2})$	$O(N^{4.2})$	$O(N^{4.2})$	$O(1)$	$O(L_{\text{Dec}})$
Query (L_{Dec})	∞	Bounded	∞	∞	Bounded
Sim. Sec.	Yes	— [‡]	Yes	Yes	Yes
Module-LWE	Yes	Yes	—	—	Yes(§7)
Without server	Yes	Yes	—	—	—

[†] The communication size of sending super-polynomially long FHE ciphertexts can be reduced to polynomially short size by transciphering [NLV11; BCK⁺23].

[‡] Only achieves *selective IND-CPA* [BS24], because of the attack of [PS25].

share and decryption share consist of $O(N^{4.2})$ [Val84; Gol20] (or $(\frac{T}{N})$ [BD10]) elements over $\mathbb{Z}_q^n$ and $\mathbb{Z}_q$, respectively (for LWE-based ThFHE). Hence, the size of the noise that appears at the decryption also becomes $O(N^{4.2})$; the modulus needs to be $\Omega(N^{4.2})$ to satisfy correctness. A simple and efficient method for achieving threshold decryption is Shamir secret sharing [Sha79], whose share size is $O(1)$ w.r.t. N . Although [BGG⁺18, Constr. 5.11] presents a ThFHE scheme with Shamir secret sharing, it requires superexponentially large modulus, e.g., $\omega((N!)^3)$ because the Lagrange coefficients are not small and blow up the noise. Hence, as noted in [PS25], the following open question remains.

Question 1.2. Can we construct threshold FHE with polynomially short decryption shares from Shamir secret sharing (instead of $\{0, 1\}$ -LSSS)?

1.1 Our Contributions

In this paper, we address both Question 1.1 and Question 1.2. We provide a brief summary of our contributions (and subsequent future works) in Table 1.

Shamir-secret-sharing-friendly. We first solve Question 1.2 in Sect. 5: we propose a knLWE-based threshold FHE scheme with Shamir secret sharing by modifying the ThFHE scheme of [PS25] via a simple masking technique. Owing to the efficiency of Shamir secret sharing, our scheme becomes compact

($= O(1)$ w.r.t. N); the sizes of moduli (Q and q), decryption shares, and secret shares do not increase depending on N . Note that, although [BGG⁺18, Constr. 8.35] also presents a compact ThFHE, it relies on a heavy cryptographic tool called an *universal thresholdizer*, which requires (non-compact) ThFHE and NIZK. In contrast, we construct a compact ThFHE scheme directly from LWE.

From standard LWE/MLWE. Next, we solve Question 1.1 in Sect. 6 and Sect. 7: we modify our knLWE-based ThFHE (presented in Sect. 5) to present ThFHE schemes from standard LWE/MLWE in Sect. 6/Sect. 7. Our core technique is what we call *noise padding*, which adds a small noise ζ whose standard deviation is $\sqrt{B_{\text{pub}} - \|\mathbf{s}\|}$ and adjusts the standard deviation of the noise obtained after decryption to be a public value composed of B_{pub} ; we thereby prevent the leakage of $\|\mathbf{s}\|$. With this technique, we construct ThFHE without knLWE and prove its security directly from standard LWE. Furthermore, we extend our LWE-based ThFHE scheme to a ThFHE scheme from MLWE.

Solely discrete. As a minor contribution, our ThFHE schemes are constructed solely with discrete values, while [PS25] requires (flooding) noise sampled from *continuous* Gaussian distribution. In general, when we implement continuous values, we need to approximate them with, e.g., floating-point numbers. However, it needs to be formally proven that the rounding errors incurred by the approximation do not affect the security and correctness. In contrast, we use *discrete* Gaussian (flooding) noise instead of continuous Gaussian noise and formally analyse correctness and security.

1.2 Future Work

Our scheme from standard (M)LWE (presented in Sect. 6 and Sect. 7) needs a bound on the number of decryption queries, as with [BS23]. We need to generate fresh padding noises ζ as many as the decryption query bound L_{Dec} , and distribute their shares to the parties at the setup stage. As a result, the storage cost for the secret shares linearly increases w.r.t. L_{Dec} . However, we note that the value of L_{Dec} is independent of any other security-related parameters, unlike in [BS23]. Thus, this drawback only increases the storage cost for the secret shares. Hence, this limitation would not be problematic for some use cases in which the number of decryption queries is small or the parties can tolerate a storage cost of $O(L_{\text{Dec}})$. We leave it an open problem to improve our scheme by removing the bound on L_{Dec} . Note that our ThFHE presented in Sect. 5 does not have this drawback since it does not use padding noise but rather relies on knLWE.

In addition, as in the prior work [PS25], our ThFHE schemes requires a party called **Server** who is untrusted (semi-honest) but assumed not to be corrupted by the adversary. This is because the ThFHE scheme of [PS25] and our schemes sanitize the input ciphertext via the **ServerDec** procedure (explained later in Sect. 2.2), using the private randomness of **Server** that remains unknown to all parties and the adversary. Let us now describe a potential idea to eliminate

the need of `Server` in ThFHE of [PS25]; we perform `ServerDec` in a distributed manner. At the first round, T parties perform `ServerDec` on the input ciphertext ct of plaintext μ using their private randomness, and broadcast the output ciphertext $\overline{\text{ct}}_i$ to the parties. At the second round, the parties calculate $\overline{\text{ct}} := \sum_i \overline{\text{ct}}_i$, whose plaintext becomes $T \cdot \mu$; we now assume $\mu \in \{0, 1\}$ and the plaintext space is $\mathbb{Z}_t$ for some $t > N$ (or, if we assume T is odd, we can use $t = 2$ and obtain $T \cdot \mu = \mu \bmod 2$). The parties then generate the partial decryption shares of $\overline{\text{ct}}$ and complete the decryption as in ThFHE of [PS25] to recover $T \cdot \mu$, then decode it to μ with division by T . In this distributed `ServerDec`, the ciphertext is successfully sanitized since $\overline{\text{ct}}$ includes randomness of honest parties. However, this idea requires one more round (for distributed `ServerDec`) than prior ThFHE schemes without `Server` such as [AJL⁺12; BGG⁺18; BS23], which makes the security proof challenging. We leave the formal concrete construction and the security proof of this potential idea to future work.

1.3 Changelog

This paper has been updated multiple times since its initial appearance on the IACR Cryptology ePrint Archive. We briefly summarize the changes below.

- Ver.1 (2025-03-04): Initial version.
- Ver.2 (2025-07-03): Minor revision.
- Ver.3 (2025-09-24): The full version of [OT26], with full proofs and omitted definitions included in the appendices.
- Ver.4 (2026-03-26) (This version): While Ver.3 required the modulus q to be even for correctness and security, we remove this restriction in Ver.4 by performing a more refined analysis of the ciphertext noise. Specifically, we improve lemmas and theorems for noise analysis (Lems. 5.1, 6.1 and 7.1), correctness (Thms. 5.3, 6.2 and 7.4), and security (Thms. 5.5, 6.3 and 7.5) by eliminating the restriction on q at the cost of only a very slight³ increase in the Gaussian parameter σ .

2 Technical Overview

We first recall the ThFHE schemes of [AJL⁺12; BGG⁺18], which are constructed (solely) with a super-polynomially large modulus in Sect. 2.1. Then, in Sect. 2.2, we describe the prior work [PS25], which achieves a polynomially small modulus for the decryption share, and we explain why their scheme needs known-norm variant of LWE. Then, Sect. 2.3, we provide a technical overview of our first ThFHE scheme, which can be efficiently constructed with Shamir secret sharing. Finally, in Sect. 2.4, we provide a technical overview of our second ThFHE scheme, which can be (directly) based on standard LWE/MLWE.

³ E.g., while $\overline{\sigma}_{\text{dec}} := \sqrt{n(\sigma_0 B_{\text{lwe}})^2 + (\sigma_{\text{flood}}/p)^2 + \sigma_1^2}$ is sufficient for [OT26, Thm. 5.3], Thm. 5.3 in this version requires $\overline{\sigma}_{\text{dec}} := \sqrt{n(\sigma_0 B_{\text{lwe}})^2 + (\sigma_{\text{flood}}/p)^2} + \sigma_1$.

2.1 Noise Flooding [AJL⁺12; BGG⁺18]

We first recall the threshold decryption procedure of the ThFHE schemes of [AJL⁺12; BGG⁺18]. For simplicity, we assume the underlying FHE is based on LWE. Let the public key be $\mathbf{pk} := (\mathbf{A}, \mathbf{b} = \mathbf{As} + \mathbf{e})$, where $\mathbf{A} \leftarrow \mathcal{U}(\mathbb{Z}_Q^{m \times n})$, $\mathbf{s} \in \mathbb{Z}_Q^n$ is a secret key, $\mathbf{e} \in \mathbb{Z}_Q^m$ is a (short) secret error, and Q is a modulus. We assume that ciphertexts, after some homomorphic evaluation, are in the following form:

$$\mathbf{ct}_{\text{eval}} := (\mathbf{a} \sim \mathcal{U}(\mathbb{Z}_Q^n), b := \mathbf{a}^\top \mathbf{s} + e_{\text{eval}} + \lfloor \frac{Q}{2} \rfloor \cdot \mu \bmod Q), \quad (1)$$

where e_{eval} is a (small) noise and $\mu \in \{0, 1\}$ is the plaintext. For simplicity, we now only consider N -out-of- N threshold decryption with additive secret sharing: each party $P_1, \dots, P_N$ holds a secret key share $\mathbf{s}_1, \dots, \mathbf{s}_N$ such that $\sum_{i=1}^N \mathbf{s}_i = \mathbf{s}$. Given $\mathbf{ct}$, each party computes a partial decryption share as follows;

$$\mathbf{pd}_i \leftarrow \text{PartDec}(\mathbf{ct}, \mathbf{s}_i) := \mathbf{a}^\top \mathbf{s}_i + e_i, \quad (2)$$

and broadcasts $\mathbf{pd}_i$ to all parties. Given $\{\mathbf{pd}_i\}_{i \in [N]}$, each party computes

$$\begin{aligned} \bar{\mu} &:= \text{FinDec}(\mathbf{ct}, \{\mathbf{pd}_i\}_{i \in [N]}) := \lfloor (b - \sum_{i \in [N]} \mathbf{pd}_i) / \lfloor \frac{Q}{2} \rfloor \rfloor \\ &= \mu + \lfloor (e_{\text{eval}} - \sum_{i \in [N]} e_i) / \lfloor \frac{Q}{2} \rfloor \rfloor \end{aligned}$$

and then correctly recovers μ if the parameters are selected so that the noise term $\bar{e} := e_{\text{eval}} - \sum_{i=1}^N e_i$ satisfies $|\bar{e}| < \lfloor \frac{Q}{4} \rfloor$ (with overwhelming probability).

Notably, each party can also recover the noise term by calculating

$$e_{\text{leak}} := b - \sum_{i \in [N]} \mathbf{pd}_i - \lfloor \frac{Q}{2} \rfloor \cdot \bar{\mu} = e_{\text{eval}} - \sum_{i \in [N]} e_i. \quad (3)$$

The noise term e_{eval} of the ciphertext after homomorphic evaluations may strongly depend on the input plaintexts. For example, in the CKKS FHE scheme [CHK⁺18], the noise term of the multiplication of two ciphertexts encrypting μ_0 and μ_1 with noise terms e_0 and e_1 is as follows:

$$e_{\text{eval}} = \mu_0 e_1 + \mu_1 e_0 + e'.$$

More simply, even without homomorphic evaluation, the ciphertext noise contains secret information; e.g., the ciphertext noise of Regev PKE [Reg09] has the form of $e_{\text{eval}} = \mathbf{r}^\top \mathbf{e}$, where $\mathbf{r}$ is a short vector and $\mathbf{e}$ is the secret error vector of the LWE public key $\mathbf{pk} := (\mathbf{A}, \mathbf{b} = \mathbf{As} + \mathbf{e})$.

This leak of the noise e_{eval} in the ciphertext is not problematic in single-key FHE (or PKE) since the decryption is performed only by the party who owns the secret key $\mathbf{s}$ (and $\mathbf{e}$) and is allowed to recover plaintexts. However, it is problematic in threshold FHE since the decryption is executed by parties who are only permitted to have the share of the secret key $\mathbf{s}_i$ and the input and result plaintexts (excluding the input plaintexts of other parties). Hence, we need to hide undesirable information in e_{eval} for security.

Asharov et al. [AJL⁺12, Lemma 2.1] presented a simple method to “smudge out” any information in e_{eval} , known as “noise flooding”, by adding super-polynomially large noise to the partial decryption share. We rewrite (2) as

$$\text{pd}_i \leftarrow \text{PartDec}(\text{ct}, \mathbf{s}_i) := \mathbf{a}^\top \mathbf{s}_i + \mathfrak{E}_i, \text{ where } |\mathfrak{E}_i| = n^{\omega(1)} \cdot B_{\text{eval}},$$

and we assume $|e_{\text{eval}}| < B_{\text{eval}}$. Then, as in (3), the recovered noise is

$$e_{\text{leak}} := b - \sum_{i \in [N]} \text{pd}_i - \lfloor \frac{Q}{2} \rfloor \cdot \bar{\mu} = e_{\text{eval}} - \sum_{i \in [N]} \mathfrak{E}_i \approx_s - \sum_{i \in [N]} \mathfrak{E}_i,$$

which information-theoretically hides e_{eval} (see Fig. 1 for graphical explanation). A notable drawback is that we need a super-polynomially large modulus Q to satisfy correctness. Thus, the size of ciphertexts, keys and shares also become super-polynomially large.

2.2 Flood-and-round [PS25]

Passelègue and Stehlé [PS25] also use noise flooding to smudge out the noise e_{eval} to prevent the leakage of information. Thus, their ThFHE scheme also requires a super-polynomially large modulus Q for the ciphertexts. However, they switch the modulus Q to a polynomially small modulus q by rounding the ciphertexts, thereby obtaining small decryption shares. This “flood-and-round”⁴ procedure, denoted by **ServerDec**, takes a super-polynomially long ciphertext (after homomorphic evaluation) ct_{eval} as input, and outputs a sanitized (by noise flooding) and rounded polynomially short ciphertext ct_{dec} :

$$\text{ServerDec}(\text{ct}_{\text{eval}} \in \mathbb{Z}_Q^{n+1}) \rightarrow \text{ct}_{\text{dec}} \in \mathbb{Z}_q^{n+1}.$$

ServerDec is executed by a special party called **Server**, who is untrusted (semi-honest) but assumed not to be corrupted by the adversary. This is because the ciphertext sanitization of **ServerDec** requires a (private) randomness of **Server**. The threshold decryption procedure is performed only on the output of **ServerDec** ct_{dec} . This is why it is called **ServerDec**: it is a part of the decryption process.

We describe the details of **ServerDec**. On the input $\text{ct}_{\text{eval}} := (\mathbf{a}, b)$ (defined as in (1)), it generates $\text{ct}_{\text{flood}} := \text{ct}_{\text{eval}} + (0, \mathfrak{E})$, where $\mathfrak{E}$ is a super-polynomially large (Gaussian) flooding noise. Then, by the smudging lemma for Gaussian [PS25, Lemma 2.1] (Lem. 3.12), we obtain

$$\begin{aligned} \text{ct}_{\text{flood}} &:= (\mathbf{a}, \mathbf{a}^\top \mathbf{s} + e_{\text{eval}} + \mathfrak{E} + \lfloor \frac{Q}{2} \rfloor \cdot \mu) \\ &\approx_s (\mathbf{a}, \mathbf{a}^\top \mathbf{s} + \mathfrak{E} + \lfloor \frac{Q}{2} \rfloor \cdot \mu). \end{aligned}$$

⁴ Passelègue and Stehlé [PS25] call their scheme “double-flood-and-round”, as they note that one more rounding can be performed on the partial decryption shares. However, they also mention that they do not use this for security. Thus, we simply call their method “flood-and-round”.

Next, the ciphertext with a large modulus Q , constructed as $Q = p \cdot q$ for $p = \Omega(2^\kappa)$ and $q = \text{poly}(\kappa)$, is rounded to a ciphertext with small modulus q :

$$\text{ct}_{\text{dec}} := \left(\left\lfloor \frac{1}{p} \cdot \mathbf{a} \right\rfloor_{\sigma_0}, \left\lfloor \frac{1}{p} \cdot (\mathbf{a}^\top \mathbf{s} + \mathfrak{E} + \lfloor \frac{Q}{2} \rfloor \cdot \mu) \right\rfloor_{\sigma_1} \right) := (\bar{\mathbf{a}}, \bar{b}),$$

where $\lfloor \cdot \rfloor_\sigma$ denotes a randomized Gaussian rounding (Def. 3.13) and σ_0, σ_1 are public parameters. Then, we have

$$\text{ct}_{\text{dec}} = (\bar{\mathbf{a}}, \bar{b} = \bar{\mathbf{a}}^\top \mathbf{s} + \mathfrak{e} + \lfloor \frac{Q}{2} \rfloor \cdot \mu \bmod q), \quad (4)$$

$$\text{where } \mathfrak{e} = \left\lfloor \frac{1}{p} \cdot (-\mathbf{r}^\top \mathbf{s} + \mathfrak{E}) \right\rfloor_{\sigma_1} \approx_s D_{\mathbb{Z}, \sqrt{\sigma_0^2 \|\mathbf{s}\|^2 + \sigma_1^2}} \quad (5)$$

and $\mathbf{r} := \mathbf{a} - p \cdot \bar{\mathbf{a}}$ is a rounding error. The partial decryption of [PS25] is $\text{pd}_i := \mathbf{a}^\top \mathbf{s}_i + e_i$ as in (2). Thus, the parties can recover the noise term as follows:

$$e_{\text{leak}} := \bar{b} - \sum_{i \in [N]} \text{pd}_i - \lfloor \frac{Q}{2} \rfloor \cdot \bar{\mu} = \mathfrak{e} - \sum_{i \in [N]} e_i.$$

However, $\mathfrak{e}$ depends on $\|\mathbf{s}\|$ as shown in (5). Because of this, [PS25] relies on the *known-norm* variant of LWE (Def. 3.15). Although known-norm LWE is reduced from standard LWE, its ring variant, known-covariance ring-LWE, has no reduction from standard assumptions such as RLWE. Thus, [PS25] noted it is an open question to extend their scheme to the ring- or module- setting.

2.3 Our Solution 1: Masking Partial Decryption

Before solving Question 1.1, we first solve Question 1.2 in Sect. 5: we propose a knLWE-based threshold FHE scheme with Shamir secret sharing by modifying ThFHE of [PS25] via simple masking technique.

Difficulty of applying Shamir secret sharing to ThFHE. Our idea of masking the partial decryption is inspired by the threshold signature scheme of [PKM⁺24]. They use a masking technique, which is also used for constructing secure aggregation protocol, e.g., [BBG⁺20; BGL⁺23], to address the difficulty of handling Lagrange coefficients in lattice-based threshold signatures. Our construction demonstrates that masking techniques can also be applied to threshold (fully homomorphic) encryption.

We first explain the difficulty of handling Lagrange coefficients in threshold (homomorphic) encryption. The ThFHE with Shamir secret sharing of [BGG⁺18, Constr. 5.11] constructs the partial decryption in the form as in (2), and the Lagrange coefficient is used in FinDec;

$$\text{FinDec}(\text{ct}, \{\text{pd}_i\}_{i \in \text{act}}) := \sum_{i \in \text{act}} \lambda_{\text{act}, i} \cdot \text{pd}_i = \mathbf{a}^\top \mathbf{s}_i + \sum_{i \in \text{act}} \lambda_{\text{act}, i} \cdot e_i \bmod q,$$

where $\text{act} \subseteq [N]$ is the set of active parties in the decryption session s.t. $|\text{act}| \geq T$. However, this requires $q = \Omega((N!)^2)$ to ensure that the large noise term

$\sum_{i \in \text{act}} \lambda_{\text{act},i} \cdot e_i$ relatively small to q [ABV⁺12]. One naive attempt to address this inefficiency is to include the Lagrange coefficient in the partial decryption:

$$\text{pd}_i \leftarrow \text{PartDec}(\text{ct}, \mathbf{s}_i, \text{act}) := \lambda_{\text{act},i} \cdot \mathbf{a}^\top \mathbf{s}_i + e_i \bmod q. \quad (6)$$

While correctness is satisfied, there is a trivial attack on this construction since the adversary can choose act to craft the coefficients $\lambda_{\text{act},i}$, and e_i is relatively small compared to q . Let $\text{act} \subseteq [N]$ s.t. $\hat{\lambda} := \lambda_{\text{act},i}$ satisfies $q = \hat{\lambda} \cdot q'$ for some integer $q' < q$, and assume $|e_i| < \hat{\lambda}$ holds. Then, we have $b'_i := \lfloor \frac{\text{pd}_i}{\hat{\lambda}} \rfloor \bmod q' = \mathbf{a}^\top \mathbf{s}_i \bmod q'$. By querying a sufficient number of such b'_i , the lower bits of the secret share $(\mathbf{s}_i \bmod q')$ can be recovered via simple linear algebra.

Our construction. We explain the construction of our Shamir-secret-sharing-friendly knLWE-based ThFHE. At the setup, each party P_i is given $\text{seed}_i \xleftarrow{\$} \{0,1\}^\kappa$ and generates pseudorandom masks by $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid})$, where PRF is (assumed to be) a pseudorandom function and sid is a session ID of the threshold decryption sessions. Then, we construct the decryption shares as:

$$\text{pd}_i \leftarrow \text{PartDec}(\text{ct}_{\text{dec}}, \mathbf{s}_i, \text{act}) := \lambda_{\text{act},i} \cdot \mathbf{a}^\top \mathbf{s}_i + \mathbf{m}_i \bmod q. \quad (7)$$

The mask value $\mathbf{m}_i$ (of any uncorrupted party P_i) is essentially pseudorandom over $\mathbb{Z}_q$ (except that the sum of all masks $\sum_{i \in \text{act}} \mathbf{m}_i$ is included in ct'_{dec} as described in (8)). Thus, the trivial attack to (6) is not valid to our construction since $\mathbf{m}_i$ is not small and pd_i is also essentially pseudorandom.

Note that we have $\sum_{i \in \text{act}} \text{pd}_i = \mathbf{a}^\top \mathbf{s} + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$ by construction of Shamir secret sharing. Hence, for correctness to be hold, the seed values $\{\text{seed}_i\}_{i \in [N]}$ are also shared with the Server: The Server also calculates $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid})$ for $i \in \text{act}$ and adds $\sum_{i \in \text{act}} \mathbf{m}_i$ to ct_{dec} in (4), then outputs:

$$\text{ct}'_{\text{dec}} := (\bar{\mathbf{a}}, \bar{b}' := \bar{\mathbf{a}}^\top \mathbf{s} + \mathbf{c} + \lfloor \frac{q}{2} \rfloor \cdot \mu + \sum_{i \in [N]} \mathbf{m}_i \bmod q). \quad (8)$$

Note that Server is untrusted (semi-honest) but assumed not to collude with the adversary, as in [PS25]; Server never receives any partial decryption, and thus knowledge of $\{\text{seed}_i\}_{i \in [N]}$ is meaningless to the untrusted Server.

Then, given $\{\text{pd}_i\}_{i \in \text{act}}$, parties perform FinDec to decrypt ct'_{dec} as follows:

$$\bar{\mu} := \lfloor (\bar{b}' - \sum_{i \in \text{act}} \text{pd}_i) / \lfloor \frac{q}{2} \rfloor \rfloor = \mu + \lfloor \mathbf{c} / \lfloor \frac{q}{2} \rfloor \rfloor.$$

The correctness holds if we select the parameters s.t. $|\mathbf{c}| < \lfloor \frac{q}{4} \rfloor$ holds (with overwhelming probability).

We now explain how to simulate the partial decryption in the security proof. The recovered noise term after decryption is

$$e_{\text{leak}} := \bar{b}' - \sum_{i \in \text{act}} \text{pd}_i - \lfloor \frac{q}{2} \rfloor \cdot \bar{\mu} = \mathbf{c} \approx_s D_{\mathbb{Z}, \sqrt{\sigma_0^2 \|\mathbf{s}\|^2 + \sigma_1^2}} \quad (9)$$

as described in (5). Let $|\text{act}| = T$ and $\text{corr} = \text{act} \setminus \{h\}$ be the set of the corrupted parties. Then, by (5), we can simulate pd_h as follows:

$$\text{pd}_h \approx_s \bar{b}' - \lfloor \frac{q}{2} \rfloor \cdot \mu - \sum_{i \in \text{corr}} \text{pd}_i + \mathfrak{e}, \text{ where } \mathfrak{e} \leftarrow D_{\mathbb{Z}, \sqrt{\sigma_0^2 \|\mathbf{s}\|^2 + \sigma_1^2}} \quad (10)$$

Note that we now rely on knLWE as in [PS25]; $\|\mathbf{s}\|$ is now public value and thus above $\mathfrak{e}$ is publicly sampleable. Given simulated pd_h instead of real one, the adversary only has information related to $\{\mathbf{s}_i\}_{i \in \text{corr}}$. Due to the security of secret sharing, no information about $\mathbf{s}_h$ and $\mathbf{s}$ is obtained from $\{\mathbf{s}_i\}_{i \in \text{corr}}$. We explain how to eliminate the need of knLWE in subsequent Sect. 2.4.

Side benefit: Compactness w.r.t. N . The recovered noise term after FinDec of the prior works [AJL⁺12; PS25] is in the form of $e_{\text{eval}} + \sum_{i \in [N]} e_i$ as in (3) (for N -out-of- N threshold decryption). Thus, its upper bound is $O(N)$ w.r.t. N , and the moduli (Q and q) should also be $\Omega(N)$ to satisfy correctness. Furthermore, for general T -out-of- N threshold decryption, this factor becomes larger, e.g., $\Omega(N^{4.2})$ or $\Omega(\frac{T}{N})$, since the recovered noise term after FinDec is in the form of $e_{\text{eval}} + \sum_{i \in S_i} e_i$, where S_i is the index set s.t. $|S_i| = O(N^{4.2})$.

In contrast, the recovered noise term after decryption of our scheme is only $\mathfrak{e}$ (or $\mathfrak{e} + \zeta$), whose size is independent on N ($=O(1)$ w.r.t. N). Thus, we achieve compactness. This is because we do not add a noise in the partial decryption share (7); instead, we add $\mathbf{m}_i$ that is canceled out to be zero after FinDec. Therefore, modulus, secret shares and decryption shares are $O(1)$ w.r.t. N .

2.4 Our Solution 2: “Noise Padding”

We next solve Question 1.1 in Sect. 6: we modify our scheme described in Sect. 2.3 to construct a ThFHE scheme from *standard* LWE. We modify the partial decryption to prevent the leakage of $\|\mathbf{s}\|$, by a core technique which we call “noise padding”.

Our construction. Our goal is to add a small noise ζ , which we call “padding noise”, to adjust the distribution of the recovered noise term e_{leak} of (9). (see Fig. 2 for a graphical explanation).

At KeyGen, we sample the padding noise ζ depending on $\mathbf{s}$ as follows:

$$\zeta \leftarrow D_{\mathbb{Z}, \sigma_{\text{padding}}}, \quad \sigma_{\text{padding}} := \sqrt{\sigma_0^2 (B_{\text{pub}}^2 - \|\mathbf{s}\|^2)},$$

where B_{pub} is a public constant (s.t. $B_{\text{pub}} > \|\mathbf{s}\|$ for any $\mathbf{s}$). Then, for $\mathfrak{e}$ in (5),

$$e_{\text{leak}} := \mathfrak{e} - \zeta \approx_s D_{\mathbb{Z}, \sigma_{\text{pub}}}, \quad \sigma_{\text{pub}} := \sqrt{\sigma_0^2 B_{\text{pub}}^2 + \sigma_1^2} \quad (11)$$

holds. Thus, if the noise term recovered after FinDec is e_{leak} as above, it does not leak any information about $\|\mathbf{s}\|$: We can construct a ThFHE scheme (directly) from standard LWE.

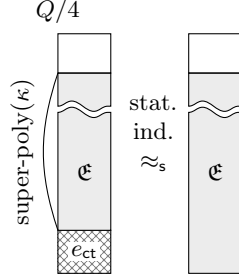


Fig. 1. Noise flooding [AJL⁺12]: Smudges out e_{eval} by adding super-polynomially large flooding noise $\mathfrak{E}$.

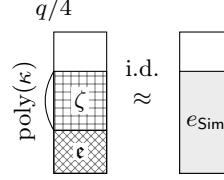


Fig. 2. Our “noise padding”: Add ζ ($V(\zeta) = \sigma^2(B_{\text{pub}}^2 - \|\mathbf{s}\|^2)$) to $\mathfrak{e}$ ($V(\mathfrak{e}) = \sigma^2\|\mathbf{s}\|^2$), obtaining the simulatable noise e_{sim} ($V(e_{\text{sim}}) = \sigma^2 B_{\text{pub}}^2$).

We now explain how to securely add ζ in the noise term recovered after FinDec. Since ζ contains information about $\|\mathbf{s}\|$, it cannot be sent directly to the untrusted (semi-honest) Server or any parties. Hence, we perform secret sharing on the padding noise ζ , as with the secret key $\mathbf{s}$: $\text{Shamir.Share}(\zeta) \rightarrow \{\zeta_1, \dots, \zeta_N\}$. Then, we modify the partial decryption of (7) by adding the shares ζ_i :

$$\text{pd}_i \leftarrow \text{PartDec}(\text{ct}_{\text{dec}}, \mathbf{s}_i, \zeta_i, \text{act}) := \lambda_{\text{act}, i} \cdot (\mathbf{a}^\top \mathbf{s}_i + \zeta_i) + \mathbf{m}_i \bmod q. \quad (12)$$

We now successfully obtain that the noise term recovered after FinDec is

$$e_{\text{leak}} := \bar{b} - \sum_{i \in \text{act}} \text{pd}_i - \lfloor \frac{q}{2} \rfloor \cdot \bar{\mu} = \mathfrak{e} - \sum_{i \in \text{act}} \zeta_i = \mathfrak{e} - \zeta.$$

Hence, as in (10), we can simulate the partial decryption shares as follows:

$$\text{pd}_h \approx_s \bar{b}' - \lfloor \frac{q}{2} \rfloor \cdot \mu - \sum_{i \in \text{corr}} \text{pd}_i + e_{\text{sim}}, \text{ where } e_{\text{sim}} \leftarrow D_{\mathbb{Z}, \sigma_{\text{pub}}},$$

where σ_{pub} is defined as in (11). Importantly, the distribution of $D_{\mathbb{Z}, \sigma_{\text{pub}}}$ contains no information about the secret key $\mathbf{s}$. Therefore, we can construct our scheme directly from *standard* LWE, without relying on known-norm LWE as in [PS25].

Limitation. Our construction needs a fresh padding noise ζ for each decryption session. We need to a priori fix the query bound $L_{\text{Dec}} = \text{poly}(\kappa)$ (as with [BS23]) and generate padding noise as many as L_{Dec} ; $\zeta := (\zeta^{(1)}, \dots, \zeta^{(L_{\text{Dec}})}) \leftarrow D_{\mathbb{Z}, \sigma_{\text{padding}}}^{L_{\text{Dec}}}$.

(Precisely, (12) is written as $\text{pd}_i := \lambda_{\text{act}, i} \cdot (\mathbf{a}^\top \mathbf{s}_i + \zeta_i^{(\text{sid})}) + \mathbf{m}_i \bmod q$.) Then, we perform Shamir secret sharing on ζ and parties need to store the shares $\{\zeta_i \in \mathbb{Z}_q^{L_{\text{Dec}}}\}_{i \in [N]} \leftarrow \text{Shamir.Share}(\zeta)$: Thus, the storage cost of each party for the shares is $O(L_{\text{Dec}})$. However, we note that the value L_{Dec} is independent of any other security-related parameters, unlike in [BS23]. Thus, this limitation would not be problematic for some use-cases in which decryption is not needed to perform many times. We leave it as an open problem to remove this limitation. Note that our previous scheme described in Sect. 2.3 (and presented in Sect. 5) does not have this limitation since the scheme does not use padding noise.

3 Preliminaries

In this section, we provide necessary definitions and lemmas. Some standard definition (of PRF) is deferred to Appendix A.

3.1 Notation

For $N \in \mathbb{N}$, we define $[N] := \{i \in \mathbb{N} \mid 1 \leq i \leq N\}$. The number of elements in a set S is denoted by $|S|$. Unless otherwise specified, we treat κ as a security parameter. We write $\text{negl}(\kappa) = \kappa^{-\omega(1)}$ for the set of negligible functions and $\text{poly}(\kappa) = \kappa^{O(1)}$ for the set of polynomial functions. We call $1 - \text{negl}(\kappa)$ overwhelming functions.

We use bold lower-case for vectors and bold upper-case for matrices. The transpose of $\mathbf{x}$ is written as $\mathbf{x}^\top$. The l_2 - and l_∞ -norm are denoted by $\|\cdot\|$ and $\|\cdot\|_\infty$, respectively. The identity matrix is denoted by $\mathbf{I}_n \in \mathbb{Z}^{n \times n}$. We write $\mathbf{\Sigma} \succ 0$ if $\mathbf{\Sigma}$ is positive definite. A square root of $\mathbf{\Sigma} \succ 0$ is a nonsingular matrix $\mathbf{S}$ such that $\mathbf{S}\mathbf{S}^\top = \mathbf{\Sigma}$, which is written as $\mathbf{S} = \sqrt{\mathbf{\Sigma}}$. The largest and smallest singular values of $\mathbf{S}$ are denoted by $\sigma_{\max}(\mathbf{S})$ and $\sigma_{\min}(\mathbf{S})$, respectively. Similarly, the largest and smallest eigenvalues are denoted by $\lambda_{\max}(\mathbf{S})$ and $\lambda_{\min}(\mathbf{S})$, respectively. We denote by $\|\mathbf{S}\|$ the matrix norm of $\mathbf{S}$ induced by the l_2 -norm. Let $\|\mathbf{S}\|_{\text{len}} = \max_{i \in [n]} \|\mathbf{s}_i\|$, where $\mathbf{s}_i$ is the i -th column vector of $\mathbf{S}$.

We denote the uniform distribution over a set S by $\mathcal{U}(S)$. For any distributions χ_1 and χ_2 , we denote by $\chi_1 + \chi_2$ the distribution $\{X_1 + X_2 \mid X_1 \leftarrow \chi_1 \text{ and } X_2 \leftarrow \chi_2 \text{ are mutually independent}\}$. We denote $X_1 \approx X_2$ if X_1 and X_2 are identically distributed. We write $\chi_1 \approx_s \chi_2$ (resp. $\chi_1 \approx_c \chi_2$) when χ_1 and χ_2 are statistically (resp. computationally) indistinguishable. For $B > 0$, we say a distribution χ over $\mathbb{R}$ is B -bounded if $\Pr_{X \leftarrow \chi}[|X| \geq B] = \text{negl}(\kappa)$.

3.2 Polynomial Ring

We denote $\mathbb{Z}_q := \mathbb{Z}/q\mathbb{Z}$ for a modulus $q \in \mathbb{N}$. We let $\mathcal{R} := \mathbb{Z}[X]/(X^n + 1)$ and $\mathcal{R}_q := \mathbb{Z}_q[X]/(X^n + 1)$ for n a power of 2. For ease of notation, we define the *coefficient vector*, *coefficient matrix* and *coefficient Gram matrix* as follows:

Definition 3.1. Let $a = \sum_{i=0}^{n-1} a_i X^i \in \mathcal{R}$, and define the coefficient vector of a as $\mathbf{a} := \text{vec}(a) := (a_0, a_1, \dots, a_{n-1})^\top \in \mathbb{Z}^n$. Let $\mathbf{P} := \begin{pmatrix} \mathbf{0} & -1 \\ \mathbf{I}_{n-1} & \mathbf{0} \end{pmatrix} \in \mathbb{Z}^{n \times n}$ be a (negacyclic) permutation matrix, and define the coefficient matrix of a as $\mathbf{A} := \text{mat}(a) := (\mathbf{a} \mathbf{P} \mathbf{a} \dots \mathbf{P}^{n-1} \mathbf{a}) \in \mathbb{Z}^{n \times n}$. The coefficient Gram matrix of a is defined as $\mathbf{\Sigma}_a := \text{Gram}(a) := \mathbf{A} \mathbf{A}^\top \in \mathbb{Z}^{n \times n}$.

For $a \in \mathcal{R}$, we define $\|a\| := \|\text{vec}(a)\|$ and $\|a\|_\infty := \|\text{vec}(a)\|_\infty$. For the distribution χ over $\mathbb{Z}^n$, we define $\mathcal{R}(\chi) := \{a \in \mathcal{R} \mid \text{vec}(a) \sim \chi\}$. Note that the coefficient matrix is useful for describing a product:

Fact 3.2. For $r, e \in \mathcal{R}$, we have $\text{vec}(re) = \mathbf{R}e = \mathbf{E}r$, where $\mathbf{R} := \text{mat}(r)$, $\mathbf{r} := \text{vec}(r)$, $\mathbf{E} := \text{mat}(e)$ and $\mathbf{e} := \text{vec}(e)$.

3.3 Lattices and Gaussians

A lattice $\mathcal{L}$ is the set of all integer linear combinations of linearly independent vectors $\mathbf{b}_1, \dots, \mathbf{b}_n \in \mathbb{R}^m$, i.e., $\mathcal{L} = \{\sum_{i=1}^n z_i \mathbf{b}_i \mid \mathbf{z} \in \mathbb{Z}^n\}$. The rank of this lattice is n and its dimension is m . If $n = m$, then the lattice is called full rank. If we arrange the vectors $\mathbf{b}_i$ as the columns of a matrix $\mathbf{B} \in \mathbb{R}^{m \times n}$, then we can write $\mathcal{L} := \mathcal{L}(\mathbf{B}) = \{\mathbf{B}\mathbf{z} \mid \mathbf{z} \in \mathbb{Z}^n\} = \mathbf{B} \cdot \mathbb{Z}^n$. For arbitrary $\mathbf{c} \in \mathbb{R}^m$, a coset of lattice $\mathcal{L}$ is defined as $A := \mathcal{L} + \mathbf{c} := \{\mathbf{v} + \mathbf{c} \mid \mathbf{v} \in \mathcal{L}\}$. The dual of a lattice $\mathcal{L}$ is $\hat{\mathcal{L}} := \{\mathbf{x} \mid \forall \mathbf{y} \in \mathcal{L}, \langle \mathbf{x}, \mathbf{y} \rangle \in \mathbb{Z}\}$. For the n -rank lattice $\mathcal{L}$ and $i \in [n]$, the successive minimum $\tilde{\lambda}_i(\mathcal{L})$ is defined as the radius of the smallest ball that contains i linearly independent vectors in $\mathcal{L}$.

For a rank- n matrix $\mathbf{S} \in \mathbb{R}^{n \times m}$, the Gaussian function on $\mathbb{R}^n$ with the (scaled) covariance matrix $\mathbf{\Sigma} = \mathbf{S}\mathbf{S}^\top \in \mathbb{R}^{n \times n}$ and a center $\mathbf{c} \in \mathbb{R}^n$ is defined as $\rho_{\mathbf{S}, \mathbf{c}}(\mathbf{x}) := \exp(-\pi(\mathbf{x} - \mathbf{c})^\top (\mathbf{S}\mathbf{S}^\top)^{-1}(\mathbf{x} - \mathbf{c}))$. Since $\rho_{\mathbf{S}, \mathbf{c}}(\mathbf{x})$ is determined by $\mathbf{\Sigma}$ (and $\mathbf{c}$), we have $\rho_{\mathbf{S}, \mathbf{c}} = \rho_{\sqrt{\mathbf{\Sigma}}, \mathbf{c}}$. When $\mathbf{S} = s\mathbf{I}_n$, we write $\rho_{\mathbf{S}, \mathbf{c}}$ as $\rho_{s, \mathbf{c}}$. For any set $A \subseteq \mathbb{R}^n$, we define $\rho_{\mathbf{S}, \mathbf{c}}(A) := \sum_{\mathbf{x} \in A} \rho_{\mathbf{S}, \mathbf{c}}(\mathbf{x})$. The subscript is $\mathbf{c} = \mathbf{0}$ when omitted.

We define the discrete Gaussian distribution over the lattice $\mathcal{L}$ as follows:

Definition 3.3 (Discrete Gaussian). For a full column-rank matrix $\mathbf{S}$, the discrete Gaussian distribution over a lattice $\mathcal{L}$ with a center $\mathbf{c} \in \mathbb{R}^n$ is defined as $\forall \mathbf{x} \in \mathcal{L}, D_{\mathcal{L}, \mathbf{S}, \mathbf{c}}(\mathbf{x}) = \rho_{\mathbf{S}, \mathbf{c}}(\mathbf{x}) / \rho_{\mathbf{S}, \mathbf{c}}(\mathcal{L})$. In particular, when $\mathbf{S}\mathbf{S}^\top = s^2 \mathbf{I}_n$ for some $s > 0$, we write $D_{\mathcal{L}, \mathbf{S}, \mathbf{c}}$ as $D_{\mathcal{L}, s, \mathbf{c}}$ and call it the spherical discrete Gaussian distribution. The subscript is $\mathbf{c} = \mathbf{0}$ when omitted.

The smoothing parameter of $\mathcal{L}$ is defined as $\eta_\epsilon(\mathcal{L}) = \min\{s \mid \rho_{1/s}(\hat{\mathcal{L}}) \leq 1 + \epsilon\}$ for $\epsilon > 0$. Unless otherwise specified, we set $\epsilon = \text{negl}(\kappa)$. An upper-bound of $\eta_\epsilon(\mathcal{L})$ is obtained with the successive minimum⁵ $\tilde{\lambda}_n(\mathcal{L})$:

Lemma 3.4 ([MR07, Lemma 3.3]). Define $\eta_\epsilon^+(\mathbb{Z}^n) := \sqrt{\ln(2n(1 + 1/\epsilon))}/\pi$. For any rank- n lattice $\mathcal{L}$ and any $\epsilon > 0$, we have $\eta_\epsilon(\mathcal{L}) \leq \tilde{\lambda}_n(\mathcal{L}) \cdot \eta_\epsilon^+(\mathbb{Z}^n)$.

In particular, $\eta_\epsilon(\mathbb{Z}^n) \leq \eta_\epsilon^+(\mathbb{Z}^n)$ holds; for any $\omega(\sqrt{\log n})$ function, there is $\epsilon = \text{negl}(\kappa)$ s.t. $\eta_\epsilon(\mathbb{Z}^n) \leq \omega(\sqrt{\log n})$.

The smoothing parameter is extended to matrices as follows:

Definition 3.5 ([Pei10, Definition 2.3]). Let $\mathbf{\Sigma} \succ 0$ be any positive definite matrix. For any lattice $\mathcal{L}$, we say that $\sqrt{\mathbf{\Sigma}} \geq \eta_\epsilon(\mathcal{L})$ if $1 \geq \eta_\epsilon(\sqrt{\mathbf{\Sigma}}^{-1} \mathcal{L})$.

We obtain a sufficient condition for showing $\sqrt{\mathbf{\Sigma}} \geq \eta_\epsilon(\mathcal{L})$ as follows⁶:

Fact 3.6. For any full-rank lattice $\mathcal{L}(\mathbf{B}) (= \mathbf{B}\mathbb{Z}^n)$ and $\mathbf{\Sigma} \succ 0$, we have $\sqrt{\mathbf{\Sigma}} \geq \eta_\epsilon(\mathcal{L})$ if $\sigma_{\min}(\sqrt{\mathbf{\Sigma}}) \geq \|\mathbf{B}\|_{\text{len}} \eta_\epsilon^+(\mathbb{Z}^n) (\geq \eta_\epsilon(\mathcal{L}))$.

Proof. By Lem. 3.4 and the hypothesis, we have $\eta_\epsilon(\sqrt{\mathbf{\Sigma}}^{-1} \mathcal{L}) \leq \tilde{\lambda}_n(\sqrt{\mathbf{\Sigma}}^{-1} \mathcal{L}) \eta_\epsilon^+(\mathbb{Z}^n) \leq \|\sqrt{\mathbf{\Sigma}}^{-1} \mathbf{B}\|_{\text{len}} \eta_\epsilon^+(\mathbb{Z}^n) \leq \|\sqrt{\mathbf{\Sigma}}^{-1}\| \|\mathbf{B}\|_{\text{len}} \eta_\epsilon^+(\mathbb{Z}^n) \leq 1$. $\square$

⁵ Although [GPV08, Lemma 3.1] provides a sharper bound with the Gram-Schmidt minimum, we use Lem. 3.4 for ease of analysis.

⁶ Although sharper bounds can be obtained by the successive minimum $\tilde{\lambda}_n$ or Gram-Schmidt minimum, we rely on $\|\cdot\|_{\text{len}}$ for ease of analysis.

The linear sum of discrete Gaussians is close to a discrete Gaussian:

Lemma 3.7 ([MP13, Theorem 3.3]). *Let $\mathcal{L}$ be an n -dimensional lattice, $\epsilon = \text{negl}(\kappa)$, $\mathbf{z} \neq \mathbf{0} \in \mathbb{Z}^m$, $s_i \geq \sqrt{2}\|\mathbf{z}\|_\infty \eta_\epsilon(\mathcal{L})$, $A_i := \mathcal{L} + \mathbf{c}_i$ be arbitrary cosets of $\mathcal{L}$, and $\mathbf{y}_i \leftarrow D_{A_i, s_i}$ for $i \in [m]$. Then, we have $\mathbf{y} := \sum_{i=1}^m z_i \mathbf{y}_i \approx_s D_{Y, s}$, where $Y = \text{gcd}(\mathbf{z})\mathcal{L} + \sum_{i=1}^m z_i \mathbf{c}_i$, and $s = \sqrt{\sum_{i=1}^m (z_i s_i)^2}$.
In particular, if $\text{gcd}(\mathbf{z}) = 1$ and $\sum_{i=1}^m z_i \mathbf{c}_i \in \mathcal{L}$, then $\mathbf{y} \approx_s D_{\mathcal{L}, s}$.*

More generally, the sum of two ellipsoid discrete Gaussians is statistically close to an ellipsoid discrete Gaussian:

Lemma 3.8 ([GMPW20, Thm. 3]). *Let $\epsilon = \text{negl}(\kappa)$. Let A_1 and A_2 be cosets of full-rank lattices $\mathcal{L}_1$ and $\mathcal{L}_2$, respectively. Let $\Sigma_1, \Sigma_2 \succ 0$ be positive definite matrices and define $\Sigma_3 := (\Sigma_1^{-1} + \Sigma_2^{-1})^{-1}$. Let $X := \{(\mathbf{x}_1, \mathbf{x}_2) \mid \mathbf{x}_1 \leftarrow D_{A_1, \sqrt{\Sigma_1}}, \mathbf{x}_2 \leftarrow \mathbf{x}_1 + D_{A_2 - \mathbf{x}_1, \sqrt{\Sigma_2}}\}$. If $\sqrt{\Sigma_2} \geq \eta_\epsilon(\mathcal{L}_2)$ and $\sqrt{\Sigma_3} \geq \eta_\epsilon(\mathcal{L}_1)$ hold, The marginal distribution of $\mathbf{x}_2$ in X is statistically close to $D_{A_2, \sqrt{\Sigma_1 + \Sigma_2}}$.*

In particular, when $A_1 \subseteq A_2$, we can simplify the above lemma:

Corollary 3.9. *Let $\epsilon = \text{negl}(\kappa)$. Let $\Sigma_1, \Sigma_2 \succ 0$ be positive definite matrices and define $\Sigma_3 := (\Sigma_1^{-1} + \Sigma_2^{-1})^{-1}$. Let A_1 and A_2 be cosets of full-rank lattices $\mathcal{L}_1$ and $\mathcal{L}_2$ such that $A_1 \subseteq A_2$. If $\sqrt{\Sigma_2} \geq \eta_\epsilon(\mathcal{L}_2)$ and $\sqrt{\Sigma_3} \geq \eta_\epsilon(\mathcal{L}_1)$ hold, we have $D_{A_1, \sqrt{\Sigma_1}} + D_{A_2, \sqrt{\Sigma_2}} \approx_s D_{A_2, \sqrt{\Sigma_1 + \Sigma_2}}$.*

The linear transformation of a discrete Gaussian is analysed as follows:

Lemma 3.10 ([GMPW20, Lemma 1]). *For any lattice coset $A = \mathcal{L} + \mathbf{c} \subseteq \mathbb{R}^n$ and matrices $\mathbf{S}, \mathbf{T}$ representing linear functions where $\mathbf{T}$ is injective on A , we have $\mathbf{T} \cdot D_{A, \mathbf{S}} = D_{\mathbf{T} \cdot A, \mathbf{T} \mathbf{S}}$. In particular, for any $s, t > 0$, we have $t \cdot D_{\mathbb{Z}, s} = D_{t\mathbb{Z}, ts}$.*

The tail bound of discrete Gaussian can be obtained as follows:

Lemma 3.11. *For any $\epsilon \in (0, 1)$, $s \geq \eta_\epsilon^+(\mathbb{Z})$ and $t \in \mathbb{N}$, we have $\Pr_{x \leftarrow D_{\mathbb{Z}, s}}[|x| \geq t] \leq \frac{s}{(1-\epsilon)\pi t} e^{-\pi t^2/s^2}$. In particular, for any $t = s \cdot \Omega(\sqrt{\kappa})$, we have $\Pr_{x \leftarrow D_{\mathbb{Z}, s}}[|x| \geq t] = \text{negl}(\kappa)$, i.e., $D_{\mathbb{Z}, s}$ is $s \cdot \Omega(\sqrt{\kappa})$ -bounded.*

Passelègue and Stehlé [PS25] presented the smudging lemma for Gaussian. We use the following lemma adapted for multivariate Gaussian:

Lemma 3.12 (Adapted from [PS25, Lemma 2.1]). *For $n = \text{poly}(\kappa)$, $\mathbf{c} \in \mathbb{R}^n$ and $\sigma = \Omega(\|\mathbf{c}\|_\infty 2^\kappa)$, we have $D_{\mathbb{Z}^n, \sigma, \mathbf{c}} \approx_s D_{\mathbb{Z}^n, \sigma}$.*

Finally, we define the randomized Gaussian rounding operation:

Definition 3.13 ($\lfloor \cdot \rfloor_\sigma$). *For any $n \in \mathbb{N}$, $\mathbf{c} \in \mathbb{R}^n$ and $\sigma > 0$, $\lfloor \mathbf{c} \rfloor_\sigma$ denotes the randomized Gaussian rounding that returns $\mathbf{y} \leftarrow D_{\mathbb{Z}^n, \sigma, \mathbf{c}}$ (or equivalently, samples $\mathbf{z} \leftarrow D_{\mathbb{Z}^n - \mathbf{c}, \sigma}$ and returns $\mathbf{z} + \mathbf{c}$).*

3.4 Learning with Errors (LWE)

The LWE distribution and the LWE problem are defined as follows:

Definition 3.14 (LWE). Let n, m, q be integers and χ be an error distribution over $\mathbb{Z}_q$. The LWE distribution is defined as

$$\text{LWE}(n, m, q, \chi) := \{(\mathbf{A}, \mathbf{b} := \mathbf{A}\mathbf{s} + \mathbf{e}) \mid \mathbf{A} \leftarrow \mathcal{U}(\mathbb{Z}_q^{m \times n}), \mathbf{s} \leftarrow \chi^n, \mathbf{e} \leftarrow \chi^m\}.$$

The advantage of an algorithm $\mathcal{A}$ for solving the (decision) LWE problem is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{LWE}} = |\Pr[\mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{u} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)) = 1]|.$$

We say that LWE is hard if $\text{Adv}_{\mathcal{A}}^{\text{LWE}} = \text{negl}(n)$ for any PPT algorithm $\mathcal{A}$.

Micciancio and Suhl [MS25] introduced a variant of LWE called “known-norm” LWE, where the norm of secret key $\mathbf{s}$ (and/or the norm of error vector $\mathbf{e}$) are known to the adversary. A reduction from (standard) LWE to known-norm LWE is also shown in [MS25]. We define known-norm LWE as follows:

Definition 3.15 (Known-norm LWE). Let n, m, q be integers and χ be an error distribution over $\mathbb{Z}_q$, and let $(\mathbf{A}, \mathbf{b} := \mathbf{A}\mathbf{s} + \mathbf{e}) \leftarrow \text{LWE}(n, m, q, \chi)$. The advantage of $\mathcal{A}$ for solving the decision known-norm LWE (knLWE) is

$$\text{Adv}_{\mathcal{A}}^{\text{knLWE}} = |\Pr[\mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}, \|\mathbf{s}\|) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{u} \leftarrow \mathcal{U}(\mathbb{Z}_q^n), \|\mathbf{s}\|) = 1]|.$$

The module-LWE problem (MLWE) is defined as follows:

Definition 3.16 (Module-LWE). Let $k, m, q \in \mathbb{N}$ and χ be a distribution over $\mathcal{R}_q$. The module-LWE distribution is defined as $\text{MLWE}(k, m, q, \chi) := \{(\mathbf{A}, \mathbf{b} := \mathbf{A}\mathbf{s} + \mathbf{e}) \mid \mathbf{A} \xleftarrow{\$} \mathcal{R}_q^{m \times k}, \mathbf{s} \leftarrow \chi^k, \mathbf{e} \leftarrow \chi^m\}$. The advantage of an algorithm $\mathcal{A}$ for solving the (decision) module-LWE problem is defined as

$$\text{Adv}_{\mathcal{A}}^{\text{MLWE}} = |\Pr[\mathcal{A}(\mathbf{A}, \mathbf{A}\mathbf{s} + \mathbf{e}) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{u} \leftarrow \mathcal{U}(\mathcal{R}_q^k)) = 1]|.$$

3.5 Shamir Secret Sharing

We describe a construction of Shamir secret sharing [Sha79] as follows:

Construction 3.17. (T, N) -Shamir secret sharing for a finite field $\mathbb{K}$ is a tuple of PPT algorithms $\text{Shamir}_{\mathbb{K}, T, N} := (\text{Share}, \text{Recon})$ defined as follows:

- **Share**($s \in \mathbb{K}$): Choose a degree $T - 1$ polynomial $P \in \mathbb{K}[X]$ that satisfies $P(0) = s$ and outputs $(s_1, \dots, s_N) = (P(1), \dots, P(N)) \in \mathbb{K}^N$.
- **Recon**($\{s_i\}_{i \in S \subseteq [N]}$): Outputs $s = \sum_{i \in S} \lambda_{S,i} s_i$, where $\lambda_{S,i} = \prod_{j \in S \setminus \{i\}} \left(\frac{-j}{i-j} \right)$.

$\text{Shamir}_{\mathbb{K}, T, N}$ satisfies correctness and privacy:

- **Correctness:** For all $S \subseteq [N]$ s.t. $|S| \geq T$, $s \in \mathbb{K}$, and $(s_1, \dots, s_N) \leftarrow \text{Shamir}_{\mathbb{K}, T, N}.\text{Share}(s)$, we have $\text{Shamir}_{\mathbb{K}, T, N}.\text{Combine}(\{s_i\}_{i \in S}) = s$.
- **Privacy:** For all $S \subset [N]$ s.t. $|S| < T$, $s^{(0)}, s^{(1)} \in \mathbb{K}$, and $(s_1^{(b)}, \dots, s_N^{(b)}) \leftarrow \text{Shamir}_{\mathbb{K}, T, N}.\text{Share}(s^{(b)})$ for $b \in \{0, 1\}$, we have $\{s_i^{(0)}\}_{i \in S} \approx \{s_i^{(1)}\}_{i \in S}$.

The above Shamir secret sharing scheme naturally extends to secrets in vector form, by performing the algorithm in an elementwise manner.

Algorithm 1: LWE-based underlying FHE scheme $\text{FHE} := (\text{PP}, \text{KeyGen}, \text{Enc}, \text{Eval}, \text{Dec})$

```

PP( $1^\kappa$ )  $\rightarrow$  pp:
1 return pp := ( $n, m, Q, \chi_{\text{lwe}}, B_{\text{eval}}$ )

KeyGen():
2  $\text{sk} := \mathbf{s} \leftarrow \chi_{\text{lwe}}^n$ ,  $\text{pk} := (\mathbf{A}, \mathbf{b} := \mathbf{A}\mathbf{s} + \mathbf{e}) \sim \text{LWE}(n, m, Q, \chi_{\text{lwe}})$ 
3 Generate the evaluation key evk ▷ We omit the details of evk
4 return (evk, pk, sk)

Enc(pk,  $\mu \in \mathcal{M} := \{0, 1\}$ ):
5  $\mathbf{r} \leftarrow \chi_{\text{lwe}}^m$ ,  $\mathbf{f} \leftarrow \chi_{\text{lwe}}^n$ ,  $f \leftarrow \chi_{\text{lwe}}$ 
6 return ct := ( $\mathbf{a}, b$ ) := ( $\mathbf{r}^\top \mathbf{A} + \mathbf{f}, \mathbf{r}^\top \mathbf{b} + f + \lfloor \frac{Q}{2} \rfloor \cdot \mu$ )
   Note: PP outputs pp s.t. the noise in the ciphertext  $e_{\text{ct}} := b - \mathbf{a}^\top \mathbf{s} - \lfloor \frac{Q}{2} \rfloor \cdot \mu$ 
   satisfies  $|e_{\text{ct}}| < B_{\text{eval}}$  with overwhelming probability

Eval(evk,  $C, \text{ct}_1, \dots, \text{ct}_l$ ):
7 return  $\overline{\text{ct}} := (\overline{\mathbf{a}}, \overline{b})$  s.t.  $e_{\text{eval}} := \overline{b} - \overline{\mathbf{a}}^\top \mathbf{s} - \lfloor \frac{Q}{2} \rfloor \cdot C(\mu_1, \dots, \mu_l)$  satisfies
    $|e_{\text{eval}}| < B_{\text{eval}}$  for any  $C$ , where  $\mu_1, \dots, \mu_l$  are the plaintexts of  $\text{ct}_1, \dots, \text{ct}_l$ 

Dec(sk, ct := ( $\mathbf{a}, b$ ))  $\rightarrow \overline{\mu} \in \mathcal{M}$ :
8 return  $\overline{\mu} := \lfloor (b - \mathbf{a}^\top \mathbf{s}) / \lfloor \frac{Q}{2} \rfloor \rfloor$ 

```

4 Definitions of FHE and Threshold FHE

In this section, we define the syntax and the framework of (threshold) FHE.

4.1 Syntax and Structure of the Underlying FHE

We first define the syntax of FHE as follows:

Definition 4.1 (FHE). A fully homomorphic encryption (FHE) scheme $\text{FHE} = (\text{PP}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ for a plaintext space $\mathcal{M}$ and a ciphertext space $\mathcal{C}$ is defined as follows.

- $\text{FHE.PP}(1^\kappa) \rightarrow \text{pp}$: On input a security parameter κ , outputs a set of public parameters pp. The following algorithms implicitly take pp as an argument.
- $\text{FHE.KeyGen}(\text{pp}) \rightarrow (\text{evk}, \text{pk}, \text{sk})$: Outputs a public evaluation key evk, a public key pk and a secret key sk.
- $\text{FHE.Enc}(\text{pk}, \mu \in \mathcal{M}) \rightarrow \text{ct} \in \mathcal{C}$, $\text{FHE.Dec}(\text{sk}, \text{ct} \in \mathcal{C}) \rightarrow \overline{\mu} \in \mathcal{M}$: Have the usual syntax for public-key encryption/decryption.
- $\text{FHE.Eval}(\text{evk}, C, \text{ct}_1, \dots, \text{ct}_l \in \mathcal{C}) \rightarrow \text{ct}_{\text{eval}} \in \mathcal{C}$: On input the evaluation key evk, a function $C : \mathcal{M}^l \rightarrow \mathcal{M}$, and a set of l ciphertexts $\text{ct}_1, \dots, \text{ct}_l$ of $\mu_1, \dots, \mu_l$, outputs the result ciphertext ct_{eval} . FHE is correct, if $\text{Dec}(\text{sk}, \overline{\text{ct}}) = f(m_1, \dots, m_l)$ holds with overwhelming probability.

Then, we specify in Algo. 1 a high-level structure for the underlying FHE scheme based on LWE, which is identical to that of [PS25]. This scheme is based on the LWE-based PKE of [LP11].

Algorithm 2: Module-LWE-based underlying FHE scheme $\text{mFHE} := (\text{PP}, \text{KeyGen}, \text{Enc}, \text{Eval}, \text{Dec})$

```

PP( $1^\kappa$ )  $\rightarrow$  pp:
1 return pp := ( $n, k, m, Q, \chi_{\text{mlwe}}, B_{\text{eval}}$ )  $\triangleright \chi_{\text{mlwe}}$ : distribution over  $\mathbb{Z}$ 
   KeyGen():
2  $\text{sk} := \mathbf{s} \leftarrow \mathcal{R}(\chi_{\text{mlwe}}^n)^k$ ,  $\text{pk} := (\mathbf{A}, \mathbf{b} := \mathbf{A}\mathbf{s} + \mathbf{e}) \sim \text{MLWE}(k, m, Q, \mathcal{R}(\chi_{\text{mlwe}}^n))$ 
3 Generate the evaluation key evk  $\triangleright$  We omit the details of evk
4 return (evk, pk, sk)
   Enc(pk,  $\mu \in \mathcal{M} := \mathcal{R}_2$ ):
5  $\mathbf{r} \leftarrow \mathcal{R}(\chi_{\text{mlwe}}^n)^m$ ,  $\mathbf{f} \leftarrow \mathcal{R}(\chi_{\text{mlwe}}^n)^k$ ,  $f \leftarrow \mathcal{R}(\chi_{\text{mlwe}}^n)$ 
6 return ct := ( $\mathbf{a}, b$ ) :=  $(\mathbf{r}^\top \mathbf{A} + \mathbf{f}, \mathbf{r}^\top \mathbf{b} + f + \lfloor \frac{Q}{2} \rfloor \cdot \mu) \in \mathcal{R}_Q^{k+1}$ 
   Note: PP outputs pp s.t. the noise in the ciphertext  $e_{\text{ct}} := b - \mathbf{a}^\top \mathbf{s} - \lfloor \frac{Q}{2} \rfloor \cdot \mu$ 
   satisfies  $\|e_{\text{ct}}\|_\infty < B_{\text{eval}}$  with overwhelming probability
   Eval(evk,  $C, \text{ct}_1, \dots, \text{ct}_l$ ):
7 return  $\bar{\text{ct}} := (\bar{\mathbf{a}}, \bar{b})$  s.t.  $e_{\text{eval}} := \bar{b} - \bar{\mathbf{a}}^\top \mathbf{s} - \lfloor \frac{Q}{2} \rfloor \cdot C(\mu_1, \dots, \mu_l)$  satisfies
    $\|e_{\text{eval}}\|_\infty < B_{\text{eval}}$  for any  $C$ , where  $\mu_1, \dots, \mu_l$  are the plaintexts of  $\text{ct}_1, \dots, \text{ct}_l$ 
   Dec(sk, ct := ( $\mathbf{a}, b$ ))  $\rightarrow \bar{\mu} \in \mathcal{M}$ :
8 return  $\bar{\mu} := \lfloor (b - \mathbf{a}^\top \mathbf{s}) / \lfloor \frac{Q}{2} \rfloor \rfloor$ 

```

We construct our ThFHE scheme on the basis of this underlying FHE scheme (Algo. 1). This framework captures most known FHE schemes, except for GSW-like [GSW13] schemes. Note that we can also use ring- or module- variants of this FHE scheme, because we do not require the “known-norm” variant of LWE for security in contrast to ThFHE construction of [PS25]. We also describe a variant of the FHE scheme based on module-LWE in Algo. 2. We construct our MLWE-based ThFHE in Sect. 7 upon this underlying MLWE-based FHE scheme.

4.2 Syntax of our Threshold FHE

Passelègue and Stehlé [PS25] generalized the definition of threshold functional encryption with the additional ServerDec algorithm. We modify the definition with our additional entities; seed values for masking $\{\text{seed}_i\}_{i \in [N]}$, session ID sid and active party set $\text{act} \subseteq [N]$. Hence, we formally describe the syntax of our threshold FHE as follows:

Definition 4.2 (Threshold FHE). Let $P_1, \dots, P_N$ be parties and $T(\leq N)$ be a threshold. Let $\mathcal{M}$ denote the plaintext space, $\mathcal{C}$ ciphertext space and $\mathcal{C}_{\text{dec}}$ the processed ciphertext space. A threshold FHE scheme is a tuple of PPT algorithms $\text{ThFHE} = (\text{PP}, \text{KeyGen}, \text{Enc}, \text{Eval}, \text{ServerDec}, \text{PartDec}, \text{FinDec})$ as follows:

- $\text{ThFHE.PP}(1^\kappa, 1^N) \rightarrow \text{pp}$: On input a security parameter κ and a number of parties N , outputs a set of public parameters pp. The following algorithms implicitly take pp as argument.

- $\text{ThFHE.KeyGen}(T, N) \rightarrow (\text{evk}, \text{pk}, \text{sk}, \{\text{sk}_i, \text{seed}_i\}_{i \in [N]}):$ On input a threshold T and a number of parties N , outputs a public evaluation key evk , a public key pk and a secret key sk , and secret shares $\{\text{sk}_i\}_{i \in [N]}$, as well as the seed value $\{\text{seed}_i\}_{i \in [N]}$.
- $\text{ThFHE.Enc}(\text{pk}, \mu \in \mathcal{M}) \rightarrow \text{ct} \in \mathcal{C}$ and $\text{ThFHE.Eval}(\text{evk}, C, \text{ct}_1, \dots, \text{ct}_l) \rightarrow \text{ct}_{\text{eval}} \in \mathcal{C}$: Same as FHE.Enc and FHE.Eval in Def. 4.1.
- $\text{ThFHE.ServerDec}(\text{pk}, \text{ct}, \text{act}, \{\text{seed}_i\}_{i \in \text{act}}) \rightarrow (\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}, \text{sid}):$ It has an internal counter sid of session ID, which is initialized $\text{sid} \leftarrow 0$ at setup. On input the public key pk , a ciphertext $\text{ct} \in \mathcal{C}$ (or an evaluation result $\text{ct}_{\text{eval}} \in \mathcal{C}$), a set of active parties act , and the set of seed values $\{\text{seed}_i\}_{i \in \text{act}}$, outputs a processed ciphertext $\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}$ and session ID sid .
- $\text{ThFHE.PartDec}(\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}, \text{sid}, \text{act}, \text{sk}_i, \text{seed}_i) \rightarrow \text{pd}_i:$ On input a ciphertext $\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}$ (outputted by ServerDec), a secret share sk_i , seed_i , sid and act , outputs a partial decryption share pd_i related to the party P_i .
- $\text{ThFHE.FinDec}(\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}, \{\text{pd}_i\}_{i \in \text{act}}) \rightarrow \bar{\mu} \in \mathcal{M}$: On input a ciphertext $\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}$ and a set of partial decryption shares $\{\text{pd}_i\}_{i \in \text{act}}$, outputs a decryption result $\bar{\mu} \in \mathcal{M}$.

5 Our Threshold FHE from Known-Norm LWE

In this section, we solve Question 1.2 (described in Sect. 1): we present a threshold FHE scheme that can be (efficiently) constructed with Shamir secret sharing.

We first describe our construction and threshold decryption procedure in Sect. 5.1. Next, we analyse the distribution of noise which can be seen to the parties (and the adversary) after the threshold decryption in Sect. 5.2. Then, relying on the noise analysis, we prove the correctness and security of our scheme in Sect. 5.3 and Sect. 5.4, respectively.

5.1 Construction and the Threshold Decryption Procedure

We present our construction of knLWE -based ThFHE in Algo. 3. The command $\text{assert}\{x\}$ outputs $\perp$ if x is false and does nothing otherwise. Our two-round threshold decryption procedure is described as follows.

Note that, in Algo. 3, pp virtually includes the bound on the decryption query L_{Dec} , and ServerDec and PartDec assert $\text{sid} \leq L_{\text{Dec}}$ only for generality of the construction: This L_{Dec} is needed for the LWE -based ThFHE that we construct in next Sect. 6. We can now regard L_{Dec} is infinitely large ($= \infty$) since Algo. 3 does not require a priori limit on the number of decryption queries.

Setting. We assume the deterministic function $\text{PRF} : \{0,1\}^* \rightarrow \mathbb{Z}_q$ is pseudorandom function. As in [PS25], we also assume the existence of party called *Server* who is semi-honest and uncorrupted. Additionally, we assume a trusted key generation process as in [BGG⁺18, Constr. 5.11].

Input/output. Let a ciphertext ct (possibly outputted by Eval) be an input of the threshold decryption process, which is given to the *Server*. If we apply

Algorithm 3: Our threshold FHE from known-norm LWE:

ThFHE := (PP, KeyGen, Enc, Eval, ServerDec, PartDec, FinDec).

```

PP( $1^\kappa, 1^N$ ):
1 return pp := ( $T, n, m, p, q, Q = p \cdot q, \chi_{\text{lwe}}, \sigma_0, \sigma_1, \sigma_{\text{flood}}, B_{\text{eval}}, L_{\text{Dec}}$ )

KeyGen( $T, N$ ):
2 ( $\text{evk}, \text{pk}_{\text{FHE}}, \text{sk} := \text{s}$ )  $\leftarrow$  FHE.KeyGen()
3  $\text{pk} := (\text{pk}_{\text{FHE}}, \|\text{s}\|)$ 
4  $\{\text{sk}_i := \text{s}_i\}_{i \in [N]} \leftarrow \text{Shamir}_{\mathbb{Z}_q^n, T, N}.\text{Share}(\text{sk})$ 
5 for  $i \in [N]$  do  $\text{seed}_i \xleftarrow{\$} \{0, 1\}^\kappa$ 
6 return ( $\text{evk}, \text{pk}, \text{sk}, \{\text{sk}_i, \text{seed}_i\}_{i \in [N]}$ )

Enc( $\text{pk}, \mu \in \mathcal{M} := \{0, 1\}$ ):
7 return ct  $\leftarrow$  FHE.Enc( $\text{pk}_{\text{FHE}}, \mu$ ) (defined in Algo. 1)

Eval( $\text{evk}, C, \text{ct}_1, \dots, \text{ct}_l$ ):
8 return  $\text{ct}_{\text{eval}} \leftarrow$  FHE.Eval( $\text{evk}, C, \text{ct}_1, \dots, \text{ct}_l$ ) (defined in Algo. 1)

ServerDec( $\text{pk}, \text{ct} \in \mathcal{C}, \text{act}, \{\text{seed}_i\}_{i \in \text{act}}$ ):
9  $\text{sid} \leftarrow \text{sid} + 1$ , assert  $\{\text{sid} \leq L_{\text{Dec}}\}$ 
10 for  $i \in \text{act}$  do  $\text{m}_i := \text{PRF}(\text{seed}_i, \text{sid}) \in \mathbb{Z}_q$ 
11  $\text{ct}_{\text{fresh}} \leftarrow \text{Enc}(\text{pk}, 0)$ 
12  $\text{ct}_{\text{in}} := (\text{ct}_{\text{in},0}, \text{ct}_{\text{in},1}) := \text{ct} + \text{ct}_{\text{fresh}} \bmod Q$ 
13  $\text{ct}_{\text{dec},0} := \left\lfloor \frac{1}{p} \cdot \text{ct}_{\text{in},0} \right\rfloor_{\sigma_0} \bmod q$ ,  $\mathfrak{E} \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sigma_{\text{flood}}}$ 
14  $\text{ct}_{\text{dec},1} := \left\lfloor \frac{1}{p} \cdot (\text{ct}_{\text{in},1} + \mathfrak{E}) \right\rfloor_{\sigma_1} + \sum_{i \in \text{act}} \text{m}_i \bmod q$ 
15 return ( $\text{ct}_{\text{dec}} := (\text{ct}_{\text{dec},0}, \text{ct}_{\text{dec},1}) \in \mathcal{C}_{\text{dec}}, \text{sid}$ )

PartDec( $\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{\mathbf{b}}) \in \mathcal{C}_{\text{dec}}, \text{sid}, \text{act}, \text{sk}_i, \text{seed}_i$ ):
16 assert  $\{\text{sid} \leq L_{\text{Dec}}\}$ ,  $\text{m}_i := \text{PRF}(\text{seed}_i, \text{sid}) \in \mathbb{Z}_q$ 
17 return  $\text{pd}_i := \lambda_{\text{act},i} \cdot \bar{\mathbf{a}}^\top \text{s}_i + \text{m}_i \bmod q$ 

FinDec( $\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{\mathbf{b}}) \in \mathcal{C}_{\text{dec}}, \text{act}, \{\text{pd}_i\}_{i \in \text{act}}$ ):
18 assert  $\{|\text{act}| \geq T\}$ 
19 return  $\bar{\mu} := \lfloor (\bar{\mathbf{b}} - \sum_{i \in \text{act}} \text{pd}_i) / \lfloor \frac{q}{2} \rfloor \rfloor$ 

```

the threshold decryption protocol to an MPC, each party P_i sends a ciphertext ct_i of its own input plaintext μ_i to the **Server** (as an additional round before the following first round). Then, the **Server** performs homomorphic evaluation of C to $\text{ct}_1, \dots, \text{ct}_N$. The result ciphertext ct_{eval} is used as an input of the threshold decryption protocol (i.e., an input of the **ServerDec**). As an output of threshold decryption, all parties except for the **Server** obtain the plaintext of ct_{eval} .

First round. On the input ciphertext ct of large modulus Q , the **Server** performs **ServerDec** to sanitize ct with noise flooding and round it to a ciphertext ct_{dec} with a small modulus q as in [PS25]. In our scheme, the **Server** further adds

a sum of masks that corresponds to all parties in act , $\{\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid})\}_{i \in \text{act}}$ in ServerDec . Then, Server broadcasts ct_{dec} to all parties in act .

Second round. Each party P_i performs PartDec on $\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{b})$ and generates the partial decryption share $\text{pd}_i := \lambda_{\text{act}, i} \cdot \bar{\mathbf{a}}^\top \mathbf{s}_i + \mathbf{m}_i$, where $\mathbf{m}_i = \text{PRF}(\text{seed}_i, \text{sid})$. P_i then broadcasts pd_i to all parties in act . Note that all parties perform PartDec only on the ciphertexts sent by the Server , and they never send partial decryption shares to the Server , under the non-collusion assumption between Server and parties as in [PS25]; the Server is assumed not to obtain any partial decryption shares. At the end of the second round, each party P_i performs FinDec to recover the plaintext, given all partial decryption shares $\{\text{pd}_i\}_{i \in \text{act}}$.

5.2 Noise Analysis

We next analyse the distribution of noise which can be seen to the parties (and the adversary) after the threshold decryption. The following lemma is used to prove both the correctness and security of our knLWE-based ThFHE scheme:

Lemma 5.1 (Noise analysis). *Let $\epsilon := \text{negl}(\kappa)$, $p \in \mathbb{N}$, $B_{\text{lwe}}, B_{\text{eval}} \in \mathbb{R}$ and let χ_{lwe} be a B_{lwe} -bounded distribution s.t. $\mathbf{s} \leftarrow \chi_{\text{lwe}}^n$ satisfies $\gcd(\mathbf{s}) = 1$ with overwhelming probability. Let $\sigma_{\text{flood}} = \Omega(2^\kappa B_{\text{eval}})$ and $\sigma_{\text{flood}} \geq \frac{\sqrt{2}}{2} p \eta_\epsilon(\mathbb{Z})$, $\sigma_0 \geq \sqrt{2} B_{\text{lwe}} \eta_\epsilon(\mathbb{Z})$, $\sigma_1 \geq \eta_\epsilon(\mathbb{Z})$. Let $\text{ct}_{\text{dec}} := (\text{ct}_{\text{dec}, 0}, \text{ct}_{\text{dec}, 1}) \in \mathcal{C}_{\text{dec}}$ be the output of ServerDec in Algo. 3, and define $\mathfrak{e}$ as the ciphertext noise of ct_{dec} ;*

$$\mathfrak{e} := \text{ct}_{\text{dec}, 1} - \text{ct}_{\text{dec}, 0}^\top \mathbf{s} - \left\lfloor \frac{q}{2} \right\rfloor \cdot \mu - \sum_{i \in \text{act}} \mathbf{m}_i \bmod q. \quad (13)$$

Then, we have

$$\mathfrak{e} \approx_{\mathbf{s}} \left\lfloor \frac{1}{p} e_0 + \left\{ \frac{q}{2} \right\} \cdot \mu \right\rfloor_{\sigma_1}, \text{ where } e_0 \leftarrow D_{\frac{1}{2} \mathbb{Z}, \sqrt{(p \sigma_0 \|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}. \quad (14)$$

Proof. We first analyse $\text{ct}_{\text{dec}} := (\text{ct}_{\text{dec}, 0}, \text{ct}_{\text{dec}, 1})$ of Algo. 3. Let the rounding error of $\text{ct}_{\text{dec}, 0} := \left\lfloor \frac{1}{p} \cdot \text{ct}_{\text{in}, 0} \right\rfloor_{\sigma_0}$ be denoted as

$$\mathbf{r} := \text{ct}_{\text{in}, 0} - p \cdot \text{ct}_{\text{dec}, 0} \sim D_{p\{\frac{1}{p} \text{ct}_{\text{in}, 0}\} + p\mathbb{Z}^n, p\sigma_0}, \quad (15)$$

where $\{x\} := x - \lfloor x \rfloor$ is the fractional part of $x \in \mathbb{R}$. Let us denote

$$\text{ct}_{\text{in}, 1} := \text{ct}_{\text{in}, 0}^\top \mathbf{s} + \left\lfloor \frac{Q}{2} \right\rfloor \cdot \mu + e_{\text{eval}} + e_{\text{fresh}}, \quad (16)$$

where e_{eval} and e_{fresh} are the decryption noises of input ciphertext ct and ct_{fresh} . Then, by definition of $\text{ct}_{\text{dec}, 1}$, (15) and (16), we have

$$\begin{aligned} \text{ct}_{\text{dec}, 1} &:= \left\lfloor \frac{1}{p} \cdot (\text{ct}_{\text{in}, 1} + \mathfrak{E}) \right\rfloor_{\sigma_1} + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q \\ &= \text{ct}_{\text{dec}, 0}^\top \mathbf{s} + \left\lfloor \frac{1}{p} \cdot \left(\mathbf{r}^\top \mathbf{s} + \left\lfloor \frac{Q}{2} \right\rfloor \cdot \mu + e_{\text{eval}} + e_{\text{fresh}} + \mathfrak{E} \right) \right\rfloor_{\sigma_1} + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q. \end{aligned}$$

Thus, we obtain

$$\mathbf{c} = \lfloor \frac{1}{p}(\mathbf{r}^\top \mathbf{s} - \{\frac{Q}{2}\} \cdot \mu + e_{\text{eval}} + e_{\text{fresh}} + \mathfrak{E}) + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}, \quad (17)$$

where we used fact that $\lfloor \frac{Q}{2} \rfloor = p \lfloor \frac{q}{2} \rfloor + p \{\frac{q}{2}\} - \{\frac{Q}{2}\}$.

We first simplify (17) by the smudging lemma. Recall that we have $|e_{\text{eval}}|, |e_{\text{fresh}}| < B_{\text{eval}}$ (as described in Algo. 1), and $\mathfrak{E} \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sigma_{\text{flood}}}$. Also note that $\{\frac{Q}{2}\} \cdot \mu \in \{0, \frac{1}{2}\} \subset \frac{1}{2}\mathbb{Z}$ (this is why we need to sample $\mathfrak{E}$ over $\frac{1}{2}\mathbb{Z}$). Thus, by the hypothesis of $\sigma_{\text{flood}} = \Omega(2^\kappa B_{\text{eval}})$ and the smudging lemma (Lem. 3.12), we obtain

$$-\{\frac{Q}{2}\} \cdot \mu + e_{\text{eval}} + e_{\text{fresh}} + \mathfrak{E} \approx_s \mathfrak{E},$$

and thus, by (17), we obtain

$$\mathbf{c} \approx_s \lfloor \frac{1}{p}(\mathbf{r}^\top \mathbf{s} + \mathfrak{E}) + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}. \quad (18)$$

Next, we analyze the distribution of $\mathbf{r}^\top \mathbf{s} + \mathfrak{E}$. Recall (15), and let $\mathbf{c} := p \{\frac{1}{p} \mathbf{c}_{\text{in},0}\}$, then $\mathbf{c} \in p \cdot (\frac{1}{p}\mathbb{Z}^n) = \mathbb{Z}^n$ is an integer vector. Let us denote the i -th elements of $\mathbf{c}$ and $\mathbf{r}$ by c_i and r_i ; then we have $r_i \sim D_{c_i + p\mathbb{Z}, p\sigma_0}$ for $i \in [n]$. By the hypothesis, we have $p \cdot \sigma_0 \geq \sqrt{2} B_{\text{we}} \eta_\epsilon(p\mathbb{Z}) \geq \sqrt{2} \|\mathbf{s}\|_\infty \eta_\epsilon(p\mathbb{Z})$. Hence, by Lem. 3.7, we obtain $\mathbf{r}^\top \mathbf{s} \approx_s D_{\mathbf{c}^\top \mathbf{s} + p\mathbb{Z}, p\sigma_0 \|\mathbf{s}\|}$. Note that $\mathbf{c}^\top \mathbf{s} + p\mathbb{Z} \subseteq \mathbb{Z} (\subset \frac{1}{2}\mathbb{Z})$ since $\mathbf{c}^\top \mathbf{s} \in \mathbb{Z}$. In addition, by the hypothesis $\sigma_{\text{flood}} \geq \sqrt{2} p \eta_\epsilon(\frac{1}{2}\mathbb{Z})$ (and $\sigma_0 \geq \sqrt{2} \eta_\epsilon(\mathbb{Z})$), we have $((\sigma_0 \|\mathbf{s}\|)^{-2} + (\sigma_{\text{flood}}/p)^{-2})^{-1/2} \geq \frac{1}{\sqrt{2}} \min(\sigma_0 \|\mathbf{s}\|, \sigma_{\text{flood}}/p) \geq \eta_\epsilon(\frac{1}{2}\mathbb{Z})$. Thus, by Cor. 3.9, we have

$$e_0 := \mathbf{r}^\top \mathbf{s} + \mathfrak{E} \approx_s D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0 \|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}.$$

Now, we obtain (14) by (18) and the above. $\square$

Solely discrete analysis. As a minor contribution, the above analysis supports a discrete Gaussian for the flooding noise $\mathfrak{E} \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sigma_{\text{flood}}}$, whereas [PS25] requires a continuous Gaussian. Thus, our scheme is constructed solely with discrete values, which is desirable for the implementation. Approximating real values with floating point values incurs rounding errors, which requires a careful analysis of security.

5.3 Correctness

We define the correctness of our knLWE-based ThFHE scheme (Algo. 3).

Definition 5.2 (Correctness). We define $\text{Game}_{\text{ThFHE}}^{\text{correct}}$ in Algo. 4. ThFHE (Algo. 3) is correct if, for any $\kappa, N > 0, T \in [1, N], l > 0, C : \mathcal{M}^l \rightarrow \mathcal{M}, (\mu_1, \dots, \mu_l) \in M^l, \text{act} \subseteq [N]$ s.t. $|\text{act}| \geq T, L_{\text{Dec}} > 0$ and $\text{sid} \leq L_{\text{Dec}}$, we have:

$$\Pr[\text{Game}_{\text{ThFHE}}^{\text{correct}}(1^\kappa, T, N, \text{sid}, \text{act}, l, C, (\mu_1, \dots, \mu_l) = 1] \geq 1 - \text{negl}(\kappa).$$

We then show the correctness of our ThFHE scheme as follows:

Algorithm 4: Correctness game for our ThFHE

$\text{Game}_{\text{ThFHE}}^{\text{correct}}(1^\kappa, T, N, \text{sid}, \text{act}, l, C, (\mu_1, \dots, \mu_l) \in \mathcal{M}^l)$:

- 1 **assert** $\{\text{act} \subseteq [N] \text{ and } |\text{act}| \geq T \text{ and } \text{sid} \leq L_{\text{Dec}}\}$
- 2 $\text{pp} \leftarrow \text{PP}(1^\kappa, 1^N)$, $(\text{evk}, \text{pk}, \text{sk}, \{\text{sk}_i, \text{seed}_i\}_{i \in [N]}) \leftarrow \text{KeyGen}(T, N)$
- 3 **for** $j \in [l]$ **do** $\text{ct}_j \leftarrow \text{Enc}(\text{pk}, \mu_j)$
- 4 $\text{ct} \leftarrow \text{Eval}(\text{evk}, C, \text{ct}_1, \dots, \text{ct}_l)$
- 5 $(\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}, \text{sid}) \leftarrow \text{ServerDec}(\text{pk}, \text{ct}, \text{act}, \{\text{seed}_i\}_{i \in \text{act}})$
- 6 **for** $i \in \text{act}$ **do** $\text{pd}_i \leftarrow \text{PartDec}(\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}, \text{sid}, \text{act}, \text{sk}_i, \text{seed}_i)$
- 7 $\bar{\mu} \leftarrow \text{FinDec}(\text{ct}_{\text{dec}}, \{\text{pd}_i\}_{i \in \text{act}}, \text{act})$
- 8 **if** $\bar{\mu} = C(\mu_1, \dots, \mu_l)$ **then return** 1 **else return** 0

Theorem 5.3. Assume that parameters $\epsilon, p, \chi_{\text{lwe}}, B_{\text{lwe}}, B_{\text{pub}}, \sigma_{\text{flood}}, \sigma_0, \sigma_1$ are selected as in Lem. 5.1. Furthermore, let $\bar{\sigma}_{\text{dec}} := \sqrt{n(\sigma_0 B_{\text{lwe}})^2 + (\sigma_{\text{flood}}/p)^2} + \sigma_1$ and $q = \bar{\sigma}_{\text{dec}} \cdot \Omega(\sqrt{\kappa})$. Then, ThFHE (Algo. 3) is correct if the underlying FHE (Algo. 1) is correct (Def. 4.1).

Proof. Let $\text{ct}_{\text{dec}} := (\text{ct}_{\text{dec},0}, \text{ct}_{\text{dec},1}) \in \mathcal{C}_{\text{dec}}$ be the output of ServerDec in Algo. 3. Then, by Lem. 5.1, we obtain

$$\text{ct}_{\text{dec},1} \approx_s \text{ct}_{\text{dec},0}^\top \mathbf{s} + \mathbf{e} + \lfloor \frac{q}{2} \rfloor \cdot \mu + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q, \quad (19)$$

where $\mathbf{e} := \lfloor \frac{1}{p} e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$ and $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}$. Let $\text{pd}_i := \lambda_{\text{act},i} \cdot \text{ct}_{\text{dec},0}^\top \mathbf{s}_i + \mathbf{m}_i \bmod q$ for $i \in \text{act}$. Then, by the correctness of Shamir secret sharing (Constr. 3.17), we have

$$\sum_{i \in \text{act}} \text{pd}_i := \text{ct}_{\text{dec},0}^\top \mathbf{s} + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q. \quad (20)$$

Finally, by (19) and (20), we have

$$\text{FinDec}(\bar{\text{ct}}, \{\text{pd}_i\}_{i \in \text{act}}, \text{act}) = \lfloor (\text{ct}_{\text{dec},1} - \sum_{i \in \text{act}} \text{pd}_i) / \lfloor \frac{q}{2} \rfloor \rfloor \approx_s \mu + \lfloor \mathbf{e} / \lfloor \frac{q}{2} \rfloor \rfloor.$$

We now only need to analyse the bound of $\mathbf{e} := \lfloor \frac{1}{p} e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$. We rewrite $\mathbf{e}$ as follows;

$$\mathbf{e} := \frac{1}{p} e_0 + \{\frac{q}{2}\} \cdot \mu + e_1,$$

where we defined $e_1 \leftarrow \mathcal{D}_{\mathbb{Z} - \frac{1}{p} e_0 - \{\frac{q}{2}\} \cdot \mu, \sigma_1}$. By Lem. 3.10, we have $\frac{1}{p} e_0 \sim D_{\frac{1}{2p}\mathbb{Z}, \sqrt{(\sigma_0\|\mathbf{s}\|)^2 + (\sigma_{\text{flood}}/p)^2}}$. Since χ_{lwe} is B_{lwe} -bounded, we have $\|\mathbf{s}\| < \sqrt{n} B_{\text{lwe}}$ (with overwhelming probability). Hence, let $\sigma' := \sqrt{n(\sigma_0 B_{\text{lwe}})^2 + (\sigma_{\text{flood}}/p)^2}$, then $\frac{1}{p} e_0$ is $\sigma' \cdot \Omega(\sqrt{\kappa})$ -bounded by Lem. 3.11. Similarly, e_1 is $\sigma_1 \Omega(\sqrt{\kappa})$ -bounded. Also note that $|\{\frac{q}{2}\} \cdot \mu| \leq \frac{1}{2} (= O(1))$. Thus, we have $|\mathbf{e}|$ is $(\sigma_1 + \sigma') \cdot \Omega(\sqrt{\kappa})$ -bounded and $(\sigma_1 + \sigma') = \bar{\sigma}_{\text{dec}}$. Hence, by selecting $q = \bar{\sigma}_{\text{dec}} \cdot \Omega(\sqrt{\kappa})$, we have $\lfloor \mathbf{e} / \lfloor \frac{q}{2} \rfloor \rfloor = 0$ with overwhelming probability; i.e., correctness holds. $\square$

Algorithm 5: The real experiment $\text{Expt}_{\mathcal{A}, \text{Real}}$ and the ideal experiment $\text{Expt}_{\mathcal{A}, \text{Ideal}}$. Differences between $\text{Expt}_{\mathcal{A}, \text{Real}}$ and $\text{Expt}_{\mathcal{A}, \text{Ideal}}$ are highlighted.

$\text{Expt}_{\mathcal{A}, \text{Real}}(1^\kappa)$:

- 1 $\text{pp} \leftarrow \text{PP}(1^\kappa, 1^N)$, $(\text{evk}, \text{pk}, \text{sk}, \{\text{sk}_i, \text{seed}_i\}_{i \in [N]}) \leftarrow \text{KeyGen}(\text{pp}, T, N)$
- 2 $(\text{corr} \subseteq [N] \text{ s.t. } |\text{corr}| < T, \text{st}) \leftarrow \mathcal{A}_1(\text{pp}, \text{pk})$
- 3 $\text{hon} := [N] \setminus \text{corr}$, $\text{ctr}_{\text{Enc}} \leftarrow 0$, $\text{ctr}_{\text{Dec}} \leftarrow 0$, $\text{sid} \leftarrow 0$, $\text{List} \leftarrow \emptyset$
- 4 **return** $\{0, 1\} \leftarrow \mathcal{A}_2^{\text{OEnc}, \text{OEval}, \text{ODec}}(\{\text{sk}_i, \text{seed}_i\}_{i \in \text{corr}}, \text{st})$

$\text{OEnc}(\mu)$:

- 5 $\text{ct} \leftarrow \text{Enc}(\text{pk}, \mu)$, $\text{ctr}_{\text{Enc}} \leftarrow \text{ctr}_{\text{Enc}} + 1$, $\text{List}[\text{ctr}] \leftarrow (\mu, \text{ct})$

$\text{OEval}(C, (i_1, \dots, i_l))$:

- 6 **assert** $\{(i_1, \dots, i_l) \subseteq [\text{ctr}_{\text{Enc}}]\}$, **for** $j \in [l]$ **do** $(\mu_j, \text{ct}_j) \leftarrow \text{List}[i_j]$
- 7 $\mu \leftarrow C(\mu_1, \dots, \mu_l)$, $\text{ct} \leftarrow \text{Eval}(\text{evk}, C, \text{ct}_1, \dots, \text{ct}_l)$
- 8 $\text{ctr}_{\text{Enc}} \leftarrow \text{ctr}_{\text{Enc}} + 1$, $\text{List}[\text{ctr}_{\text{Enc}}] \leftarrow (\mu, \text{ct})$

$\text{ODec}(\text{act})$:

- 9 **assert** $\{|\text{act}| \geq T \text{ and } \text{ctr}_{\text{Dec}} < \text{ctr}_{\text{Enc}} \text{ and } \text{ctr}_{\text{Dec}} < L_{\text{Dec}}\}$
- 10 $\text{ctr}_{\text{Dec}} \leftarrow \text{ctr}_{\text{Dec}} + 1$, $(\mu, \text{ct}) \leftarrow \text{List}[\text{ctr}_{\text{Dec}}]$, $\text{corr}_{\text{act}} := \text{corr} \cap \text{act}$, $\text{hon}_{\text{act}} := \text{hon} \cap \text{act}$
- 11 $(\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}, \text{sid}) \leftarrow \text{ServerDec}(\text{pk}, \text{ct}, \text{act}, \{\text{seed}_i\}_{i \in \text{act}})$
- 12 $\{\text{pd}_i\}_{i \in \text{act}} \leftarrow \{\text{PartDec}(\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}, \text{sid}, \text{act}, \text{sk}_i, \text{seed}_i)\}_{i \in \text{act}}$
- 13 **return** $(\text{ct}_{\text{dec}}, \{\text{pd}_i\}_{i \in \text{act}})$

$\text{Expt}_{\mathcal{A}, \text{Ideal}}(1^\kappa)$:

Same as $\text{Expt}_{\mathcal{A}, \text{Real}}(1^\kappa)$ except for that $\{\text{sk}_i\}_{i \in [N]}$ is replaced with:

- 14 $\{\text{sk}_i\}_{i \in [N]} \leftarrow \text{Shamir.Share}_{\mathbb{Z}_q, T, N}((0, 0))$ $\triangleright$ Shares with no information
- $\triangleright$ OEnc and OEval are identical to those in $\text{Expt}_{\mathcal{A}, \text{Real}}$.

$\text{ODec}(\text{act})$:

$\triangleright$ Perform lines 9–10 of $\text{Expt}_{\mathcal{A}, \text{Real}}.\text{ODec}(\text{act})$ in Algo. 5

- 15 **return** $(\text{ct}_{\text{dec}}, \{\text{pd}_i\}_{i \in \text{act}}) \leftarrow \text{Sim}(\text{ct}, \mu, \text{pk}, \{\text{sk}_i, \text{seed}_i\}_{i \in \text{corr}}, \text{act})$

5.4 Security

We define the security of our ThFHE scheme in Def. 5.4. Informally, the definition implies that the view of the real world ($:= \text{Expt}_{\mathcal{A}, \text{Real}}$), where the adversary receives an honestly generated set of partial decryption shares

$$\{\text{pd}_i\}_{i \in \text{act}} \leftarrow \{\text{PartDec}(\text{ct}_{\text{dec}} \in \mathcal{C}_{\text{dec}}, \text{sid}, \text{act}, \text{sk}_i, \text{seed}_i)\}_{i \in \text{act}}$$

is (computationally) indistinguishable from the view of the ideal world ($:= \text{Expt}_{\mathcal{A}, \text{Ideal}}$), where the adversary receives simulated partial decryption shares

$$\{\text{pd}_i\}_{i \in \text{act}} \leftarrow \text{Sim}(\overline{\text{ct}}, \{\text{sk}_i, \text{seed}_i\}_{i \in \text{corr}})$$

generated by some PPT simulator Sim by using only incomplete secret share sets $\{\text{sk}_i, \text{seed}_i\}_{i \in \text{corr}}$ ($|\text{corr}| < t$) that are given to the adversary. Hence, owing to the privacy of secret sharing (Constr. 3.17), the secret share $\{\text{sk}_i\}_{i \in \text{corr}}$ provide

no information regarding sk to the adversary; they are simulated with the shares of meaningless zero vectors $\{\text{sk}_i\}_{i \in [N]} \leftarrow \text{Shamir.Share}(\mathbf{0})$.

Definition 5.4 (Security). ThFHE (Algo. 3) is simulation-secure if, for any PPT algorithms $\mathcal{D}$ and $\mathcal{A}$, there exists a PPT simulator Sim s.t. $\text{Adv}_{\mathcal{D}, \mathcal{A}}^{\text{ThFHE}}(\kappa) := |\Pr[\mathcal{D}(\text{Expt}_{\mathcal{A}, \text{Real}}(1^\kappa)) = 1] - \Pr[\mathcal{D}(\text{Expt}_{\mathcal{A}, \text{Ideal}}(1^\kappa)) = 1]| = \text{negl}(\kappa)$, where $\text{Expt}_{\mathcal{A}, \text{Real}}(1^\kappa)$ and $\text{Expt}_{\mathcal{A}, \text{Ideal}}(1^\kappa)$ are defined in Algo. 5.

Then, we prove the above security in Thm. 5.5.

Theorem 5.5. ThFHE (Algo. 3) is simulation-secure under the assumption on the pseudorandomness of PRF and the LWE assumption: Formally, for any PPT algorithms $\mathcal{D}$ and $\mathcal{A}$, there exists PPT algorithms Sim , $\mathcal{B}_{\text{LWE}}$ and $\mathcal{B}_{\text{PRF}}$ s.t.

$$\text{Adv}_{\mathcal{D}, \mathcal{A}}^{\text{ThFHE}}(\kappa) < \text{Adv}_{\mathcal{B}_{\text{LWE}}}^{\text{LWE}(m, n, Q, \chi_{\text{lwe}})} + \text{Adv}_{\mathcal{B}_{\text{PRF}}}^{\text{PRF}}(\kappa) + \text{negl}(\kappa),$$

if $\epsilon, p, q, Q, \chi_{\text{lwe}}, B_{\text{lwe}}, B_{\text{pub}}, \sigma_{\text{flood}}, \sigma_0, \sigma_1$ are selected as in Thm. 5.3.

Proof. We provide the full formal proof in Appendix B. Here, we provide an overview of the proof. We define hybrid experiments $\text{Hyb}_1, \dots, \text{Hyb}_6$ to show that $\text{Expt}_{\mathcal{A}, \text{Real}} \approx_c \text{Expt}_{\mathcal{A}, \text{Ideal}}$ holds for some PPT simulator Sim .

In Hyb_1 , we replace the output $\text{ct}_{\text{dec}, 1}$ of ServerDec with $\text{ct}_{\text{dec}, 1} = \text{ct}_{\text{dec}, 0}^T \mathbf{s} + \mathbf{e} + \lfloor \frac{q}{2} \rfloor \mu + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$, where $\mathbf{e} := \lfloor \frac{1}{p} e_0 + \{\frac{q}{2}\} \mu \rfloor_{\sigma_1}$ and $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}$ (as in (14)). We have $\text{Expt}_{\mathcal{A}, \text{Real}} \approx_s \text{Hyb}_1$ by Lem. 5.1.

In Hyb_2 , $\text{ct}_{\text{dec}, 0} := \lfloor \frac{1}{p} \text{ct}_{\text{in}, 0} \rfloor_{\sigma_0}$ is replaced with $\text{ct}_{\text{dec}, 0} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$. We have $\text{Hyb}_1 \approx_c \text{Hyb}_2$ by LWE assumption. Now, the output of the ServerDec is

$$\text{ct}_{\text{dec}} := (\bar{\mathbf{a}} \leftarrow \mathcal{U}(\mathbb{Z}_q^n), \bar{b} := \bar{\mathbf{a}}^T \mathbf{s} + \mathbf{e} + \lfloor \frac{q}{2} \rfloor \cdot \mu + \sum_{i \in \text{act}} \mathbf{m}_i). \quad (21)$$

In Hyb_3 , for $i \in \text{hon}_{\text{act}}$, we replace $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid})$ with $\mathbf{m}_i \leftarrow \mathcal{U}(\mathbb{Z}_q)$. We have $\text{Hyb}_2 \approx_c \text{Hyb}_3$ by the assumption on the pseudorandomness of PRF.

In Hyb_4 , we fix some $h \in \text{hon}_{\text{act}}$ and replace the decryption share of P_h with

$$\text{pd}_h := \bar{b} - \lfloor \frac{q}{2} \rfloor \cdot \mu - \sum_{i \in \text{act} \setminus \{h\}} \text{pd}_i - \mathbf{e}. \quad (22)$$

Since we have $\sum_{i \in \text{act}} \text{pd}_i = \bar{b} - \lfloor \frac{q}{2} \rfloor \cdot \mu - \mathbf{e}$ in both Hyb_3 and Hyb_4 (by (19) and (20)), we have $\text{Hyb}_3 \approx \text{Hyb}_4$ (=identically distributed).

In Hyb_5 , $\mathbf{m}_h$ is no longer sampled, and $\bar{b}$ is directly sampled uniformly random. In the previous Hyb_4 , $\mathbf{m}_h \leftarrow \mathcal{U}(\mathbb{Z}_q)$ is only used to calculate $\bar{b}$ in (21); thus, $\bar{b} \sim \mathcal{U}(\mathbb{Z}_q)$ and $\text{Hyb}_4 \approx \text{Hyb}_5$ hold. Similarly, in Hyb_6 , for $i \in \text{hon}_{\text{act}} \setminus \{h\}$, $\mathbf{m}_i$ is no longer sampled and pd_i is directly sampled uniformly random. We have $\text{Hyb}_5 \approx \text{Hyb}_6$ since $\mathbf{m}_i \sim \mathcal{U}(\mathbb{Z}_q)$ is only used to calculate pd_i in Hyb_5 .

Let Sim be the (PPT) procedure of Hyb_6 for outputting $(\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{b}), \{\text{pd}_i\}_{i \in \text{act}})$. Sim only uses the corrupted secret shares $\{\text{sk}_i, \text{seed}_i\}_{i \in \text{corr}}$ (and other public values), and $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}$ is simulatable since $\mathbf{pk}$ contains $\|\mathbf{s}\|$ by construction (Algo. 3). Hence, by the privacy of Shamir secret sharing, $\{\text{sk}_i\}_{i \in \text{corr}} \leftarrow \text{Shamir.Share}(\text{sk})$ and $\{\text{sk}_i\}_{i \in \text{corr}} \leftarrow \text{Shamir.Share}(\mathbf{0})$ are identically distributed. Thus, we obtain $\text{Hyb}_6 \approx \text{Expt}_{\mathcal{A}, \text{Ideal}}$. $\square$

The need for semantic security of the underlying FHE. Note that we do not rely on the known-norm LWE assumption for this simulation security, but rely on the standard LWE assumption. Interestingly, we do not require the semantic security of the underlying FHE for our security proof (Thm. 5.5); we use the standard LWE assumption (with ephemeral secret vector $\mathbf{r}$ for encryption) only for sanitizing ciphertexts in the `ServerDec` procedure. Intuitively, this is because the partial decryption shares are masked with pseudorandom outputs of the PRF. Our security definition (Def. 5.4) keeps the encryption oracle `Enc` unchanged in the ideal experiment, and we only show that partial decryption shares do not reveal any information to the adversary. The semantic security of underlying FHE, which can be shown from known-norm LWE, becomes necessary for applications of our threshold decryption procedure. For example, in MPC, we can use the semantic security of the underlying FHE to prove that no information about the plaintext input of the ciphertext of each party P_i is provided to `Server` (and other parties).

Security against the untrusted (semi-honest) Server. Def. 5.4 defines security against an adversary who corrupts parties. Although the `Server` is assumed not to collude with any parties, it is untrusted (semi-honest). We can prove the security against the `Server` more easily than in Thm. 5.5. Although the `Server` receives seed_i for PRF of all parties act_i , outputs of PRF ($=\mathbf{m}_i$) are used only for calculating partial decryption shares. Since `Server` does not receive any partial decryption shares under the non-collusion assumption, the knowledge of seed_i is meaningless to `Server`. Therefore, `Server` sees only input FHE ciphertexts ct (and other public entities), the semantic security of which follows from that of the underlying FHE scheme.

6 Our Threshold FHE from (Standard) LWE

In this section, we present a threshold FHE scheme, which can be (directly) constructed from (standard) LWE without using the known-norm variant of LWE as in [PS25] (and Sect. 5). This threshold FHE scheme is extended to the one from Module-LWE in Sect. 7, which solves Question 1.1.

We first describe our construction in Sect. 6.1. Next, we analyse the distribution of noise which can be seen to the parties (and the adversary) after the threshold decryption in Sect. 6.2. Then, we prove the correctness and security of our scheme in Sect. 6.3 and Sect. 6.4, respectively.

6.1 Construction

We construct our (standard-)LWE-based ThFHE in Algo. 6 based on our knLWE-based ThFHE presented in the previous section (Algo. 3). The only difference between Algo. 6 and Algo. 3 is the additional entity called “padding noise” ζ .

In Algo. 6, we first sample padding noise $\zeta^{(\text{sid})}$ in line 3 for $\text{sid} \in L_{\text{Dec}}$, where L_{Dec} is the bound on the threshold decryption query: Since we need a fresh

Algorithm 6: Our threshold FHE from (standard) LWE:
 ThFHE := (PP, KeyGen, Enc, Eval, ServerDec, PartDec, FinDec).

```

  PP( $1^\kappa, 1^N$ ):
  1 return  $\mathbf{pp} := (T, n, m, p, q, Q = p \cdot q, \chi_{\text{lwe}}, \sigma_0, \sigma_1, \sigma_{\text{flood}}, B_{\text{pub}}, B_{\text{eval}}, L_{\text{Dec}})$ 

  KeyGen( $T, N$ ):
  2  $(\text{evk}, \text{pk}, \text{sk}) \leftarrow \text{FHE.KeyGen}()$   $\triangleright \text{sk} := \mathbf{s} \leftarrow \chi_{\text{lwe}}^n$ 
  3  $\text{err} := \boldsymbol{\zeta} := (\zeta^{(1)}, \dots, \zeta^{(L_{\text{Dec}})}) \leftarrow D_{\mathbb{Z}, \sigma_0 \sqrt{B_{\text{pub}}^2 - \|\mathbf{s}\|^2}}^{L_{\text{Dec}}}$   $\triangleright$  “Padding noise”
  4  $\{\text{sk}_i := (\mathbf{s}_i \in \mathbb{Z}_q^n, \boldsymbol{\zeta}_i \in \mathbb{Z}_q^{L_{\text{Dec}}})\}_{i \in [N]} \leftarrow \text{Shamir}_{\mathbb{Z}_q, T, N}.\text{Share}((\text{sk}, \text{err}))$ 
  5 for  $i \in [N]$  do  $\text{seed}_i \xleftarrow{\$} \{0, 1\}^\kappa$ 
  6 return  $(\text{evk}, \text{pk}, \text{sk}, \{\text{sk}_i, \text{seed}_i\}_{i \in [N]})$ 

  Enc( $\text{pk}, \mu \in \mathcal{M} := \{0, 1\}$ ):
  7 return  $\text{ct} \leftarrow \text{FHE.Enc}(\text{pk}, \mu)$  (defined in Algo. 1)
   $\triangleright$  Eval, ServerDec and FinDec are identical to those of Algo. 3

  PartDec( $\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{b}) \in \mathcal{C}_{\text{dec}}, \text{sid}, \text{act}, \text{sk}_i := (\mathbf{s}_i, \boldsymbol{\zeta}_i), \text{seed}_i$ ):
  8 assert  $\{\text{sid} \leq L_{\text{Dec}}\}, \mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid}) \in \mathbb{Z}_q$ 
  9 return  $\text{pd}_i := \lambda_{\text{act}, i} \cdot (\bar{\mathbf{a}}^\top \mathbf{s}_i + \zeta_i^{(\text{sid})}) + \mathbf{m}_i \bmod q$   $\triangleright$  Add padding noise share  $\zeta_i^{(\text{sid})}$ 

```

padding noise $\zeta^{(\text{sid})}$ for each threshold decryption session, we need to sample padding noise as many as the number of queries on the threshold decryption at the setup. Shamir shares $(\zeta_1^{(\text{sid})}, \dots, \zeta_N^{(\text{sid})})$ of each padding noise $\zeta^{(\text{sid})}$ are distributed for each party as with the shares of secret keys. The threshold decryption shares of Algo. 6 includes $\zeta_i^{(\text{sid})}$ as $\text{pd}_i := \lambda_{\text{act}, i} \cdot (\bar{\mathbf{a}}^\top \mathbf{s}_i + \zeta_i^{(\text{sid})}) + \mathbf{m}_i$. The threshold decryption procedure is identical to that described in Sect. 5.1.

6.2 Noise Analysis

We next analyse the distribution of noise which can be seen to the parties (and the adversary). The only difference from Lem. 5.1 is that we now have an additional term of padding noise ζ :

Lemma 6.1 (Noise analysis). *Assume that parameters $\epsilon, p, \chi_{\text{lwe}}, B_{\text{lwe}}, \sigma_{\text{flood}}, \sigma_0, \sigma_1$ are selected as in Lem. 5.1, and define ϵ as in (13). Furthermore, let $B_{\text{pub}} \geq \sqrt{nB_{\text{lwe}}^2 + 1}$ and $\zeta \sim D_{\mathbb{Z}, \sigma_0 \sqrt{B_{\text{pub}}^2 - \|\mathbf{s}\|^2}}$, then we have*

$$\epsilon + \zeta \approx_s \lfloor \frac{1}{p} e'_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}, \text{ where } e'_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0 B_{\text{pub}})^2 + \sigma_{\text{flood}}^2}}. \quad (23)$$

Proof. By Lem. 5.1, we have

$$\epsilon \approx_s \lfloor \frac{1}{p} e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}, \text{ where } e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0 \|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}$$

as in (14). Thus, we have $\epsilon + \zeta \approx_s \lfloor \frac{1}{p} (e_0 + p \cdot \zeta) + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$. By Lem. 3.10, we have $p \cdot \zeta \sim D_{p\mathbb{Z}, p\sigma_0 \sqrt{B_{\text{pub}}^2 - \|\mathbf{s}\|^2}}$. By the hypothesis $\sigma_{\text{flood}} \geq \sqrt{2p\eta_\epsilon}(\frac{1}{2}\mathbb{Z})$ and

$\sigma_0 \geq \sqrt{2}\eta_\epsilon(\mathbb{Z})$, we have

$$\begin{aligned} & (((p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2)^{-1} + (p^2\sigma_0^2(B_{\text{pub}}^2 - \|\mathbf{s}\|^2))^{-1})^{-1/2} \\ & \geq \frac{p}{\sqrt{2}} \min(\sigma_0\|\mathbf{s}\|, \sigma_0\sqrt{B_{\text{pub}}^2 - \|\mathbf{s}\|^2}, \sigma_{\text{flood}}/p) \geq \frac{p}{\sqrt{2}} \min(\sigma_0, \sigma_{\text{flood}}/p) \geq \eta_\epsilon(p\mathbb{Z}), \end{aligned}$$

where we use the fact that $\sqrt{B_{\text{pub}}^2 - \|\mathbf{s}\|^2} \geq \sqrt{B_{\text{pub}}^2 - nB_{\text{lwe}}^2} \geq 1$. Thus, by Cor. 3.9, we have $e'_0 := e_0 + p \cdot \zeta \approx_s D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0 B_{\text{pub}})^2 + \sigma_{\text{flood}}^2}}$. $\square$

6.3 Correctness

The correctness of our LWE-based ThFHE scheme is identically defined as in Def. 5.2. We prove the correctness as follows:

Theorem 6.2 (Correctness). *Assume that parameters $\epsilon, p, \chi_{\text{lwe}}, B_{\text{lwe}}, B_{\text{pub}}, \sigma_{\text{flood}}, \sigma_0$, and σ_1 are selected as in Lem. 6.1. Furthermore, let $\sigma_{\text{dec}} := \sqrt{(\sigma_0 B_{\text{pub}})^2 + (\sigma_{\text{flood}}/p)^2} + \sigma_1$ and $q = \sigma_{\text{dec}} \cdot \Omega(\sqrt{\kappa})$. Then, ThFHE (Algo. 6) is correct (Def. 5.2) if the underlying FHE (Algo. 1) is correct (Def. 4.1).*

Proof. The proof is identical to the proof of Thm. 5.3 until (19) is reached. After that, we now have $\mathbf{pd}_i := \lambda_{\text{act},i} \cdot (\mathbf{ct}_{\text{dec},0}^T \mathbf{s}_i + \zeta_i^{(\text{sid})}) + \mathbf{m}_i \bmod q$ for $i \in \text{act}$. By the correctness of Shamir secret sharing, we have

$$\sum_{i \in \text{act}} \mathbf{pd}_i := \mathbf{ct}_{\text{dec},0}^T \mathbf{s} + \zeta^{(\text{sid})} + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q. \quad (24)$$

By (19) and (24), we have

$$\text{FinDec}(\overline{\mathbf{ct}}, \{\mathbf{pd}_i\}_{i \in \text{act}}, \text{act}) = \lfloor (\mathbf{ct}_{\text{dec},1} - \sum_{i \in \text{act}} \mathbf{pd}_i) / \lfloor \frac{q}{2} \rfloor \rfloor \approx_s \mu + \lfloor (\mathbf{e} + \zeta^{(\text{sid})}) / \lfloor \frac{q}{2} \rfloor \rfloor.$$

Hence, we only need to analyse the bound of $\mathbf{e} + \zeta^{(\text{sid})}$. By Lem. 6.1, we have

$$\mathbf{e} + \zeta^{(\text{sid})} \approx_s \lfloor \frac{1}{p} e'_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}, \text{ where } e'_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0 B_{\text{pub}})^2 + \sigma_{\text{flood}}^2}}.$$

We rewrite the above as follows;

$$\mathbf{e} + \zeta^{(\text{sid})} \approx_s \frac{1}{p} e'_0 + \{\frac{q}{2}\} \cdot \mu + e_1,$$

where we defined $e_1 \leftarrow \mathcal{D}_{\mathbb{Z} - \frac{1}{p}e'_0 - \{\frac{q}{2}\} \cdot \mu, \sigma_1}$. By Lem. 3.10, we have $\frac{1}{p}e'_0 \sim D_{\frac{1}{2p}\mathbb{Z}, \sqrt{(\sigma_0 B_{\text{pub}})^2 + (\sigma_{\text{flood}}/p)^2}}$. Hence, let $\sigma' := \sqrt{(\sigma_0 B_{\text{pub}})^2 + (\sigma_{\text{flood}}/p)^2}$, then $\frac{1}{p}e'_0$ is $\sigma' \cdot \Omega(\sqrt{\kappa})$ -bounded by Lem. 3.11. Similarly, e_1 is $\sigma_1 \Omega(\sqrt{\kappa})$ -bounded. Also note that $|\{\frac{q}{2}\} \cdot \mu| \leq \frac{1}{2} (= O(1))$. Thus, we have $|\mathbf{e} + \zeta^{(\text{sid})}|$ is $(\sigma_1 + \sigma') \cdot \Omega(\sqrt{\kappa})$ -bounded and $(\sigma_1 + \sigma') = \overline{\sigma}_{\text{dec}}$. Hence, by selecting $q = \overline{\sigma}_{\text{dec}} \cdot \Omega(\sqrt{\kappa})$, we have $\lfloor \mathbf{e} + \zeta^{(\text{sid})} / \lfloor \frac{q}{2} \rfloor \rfloor = 0$ with overwhelming probability; i.e., correctness holds. $\square$

6.4 Security

The security of our LWE-based ThFHE (Algo. 6) is also defined as in Def. 5.4, since the syntax is identical to that of knLWE-based ThFHE (Algo. 3). We now prove the security:

Theorem 6.3 (Security). *ThFHE (Algo. 6) is simulation-secure (Def. 5.4) under the assumption on the pseudorandomness of PRF and the LWE assumption: Formally, for any PPT algorithms $\mathcal{D}$ and $\mathcal{A}$, there exists PPT algorithms Sim , $\mathcal{B}_{\text{LWE}}$ and $\mathcal{B}_{\text{PRF}}$ s.t.*

$$\text{Adv}_{\mathcal{D}, \mathcal{A}}^{\text{ThFHE}}(\kappa) < \text{Adv}_{\mathcal{B}_{\text{LWE}}}^{\text{LWE}(n, m, Q, \chi_{\text{lwe}})} + \text{Adv}_{\mathcal{B}_{\text{PRF}}}^{\text{PRF}}(\kappa) + \text{negl}(\kappa),$$

if $\epsilon, p, q, Q, \chi_{\text{lwe}}, B_{\text{lwe}}, B_{\text{pub}}, \sigma_{\text{flood}}, \sigma_0, \sigma_1$ are selected as in Thm. 6.2.

Proof. The proof is almost identical to Thm. 5.5, except for the simulation of the noise revealed after decryption. Let Hyb'_6 be a variant of Hyb_6 in the proof of Thm. 5.5, where pd_h in (22) is replaced with

$$\text{pd}_h := \bar{b} - \lfloor \frac{q}{2} \rfloor \cdot \mu - \sum_{i \in \text{act} \setminus \{h\}} \text{pd}_i - \mathbf{e} + \zeta^{(\text{sid})},$$

where $\mathbf{e} := \lfloor \frac{1}{p} e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$, $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}$ as in (14) and $\zeta^{(\text{sid})} \leftarrow D_{\mathbb{Z}, \sigma_0 \sqrt{B_{\text{pub}}^2 - \|\mathbf{s}\|^2}}$. In a manner that is similar to the proof of Thm. 5.5, we obtain $\text{Expt}_{\mathcal{A}, \text{Real}} \approx_c \text{Hyb}'_6$. Furthermore, by Lem. 6.1, we obtain $-\mathbf{e} + \zeta^{(\text{sid})} \approx_s \lfloor \frac{1}{p} e'_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$, where $e'_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0 B_{\text{pub}})^2 + \sigma_{\text{flood}}^2}}$ as in (23). Thus, the challenger can simulate $-\mathbf{e} + \zeta^{(\text{sid})}$ (without knowledge of $\|\mathbf{s}\|$, as in Thm. 5.5), since it consists solely of public values. Let the PPT simulator Sim of $\text{Expt}_{\mathcal{A}, \text{Ideal}}$ (in Algo. 5) be the procedures of Hyb'_6 , then $\text{Hyb}'_6 \approx \text{Expt}_{\mathcal{A}, \text{Ideal}}$ holds by the privacy of Shamir secret sharing (Constr. 3.17). $\square$

7 Our Threshold FHE from Module-LWE

Our threshold FHE scheme (Algo. 6) in the previous section is constructed from standard LWE, without relying on known-norm LWE as in [PS25] (and Sect. 5). Thus, Algo. 6 can be naturally extended to the construction based on module-LWE. However, the distribution of the noise is nontrivially different in the MLWE setting. The padding noise is changed from the univariate Gaussian $D_{\mathbb{Z}, \sigma_0 \sqrt{B_{\text{pub}}^2 - \|\mathbf{s}\|^2}}$ in the LWE setting (Algo. 6) to the multivariate Gaussian $D_{\mathbb{Z}^n, \sigma_0 \sqrt{B_{\text{pub}}^2} \mathbf{I} - \Sigma_{\mathbf{s}}}$ in the MLWE setting (Algo. 7), where $\Sigma_{\mathbf{s}}$ is a covariance matrix that corresponds to the secret vector $\mathbf{s} \in \mathcal{R}^k$.

We first describe our construction in Sect. 7.1. Next, we provide the noise analysis in the MLWE setting in Sect. 7.2. Then, we prove the correctness and security of mThFHE in Sect. 7.3 and Sect. 7.4, respectively.

Algorithm 7: Our threshold FHE from module-LWE: $\text{mThFHE} := (\text{PP}, \text{KeyGen}, \text{Enc}, \text{Eval}, \text{ServerDec}, \text{PartDec}, \text{FinDec})$.

```

PP( $1^\kappa, 1^N$ ):
1 return  $\text{pp} := (T, n, k, m, p, q, Q = p \cdot q, \chi_{\text{mlwe}}, \sigma_0, \sigma_1, \sigma_{\text{flood}}, B_{\text{pub}}, B_{\text{eval}}, L_{\text{Dec}})$ 
KeyGen( $T, N$ ):
2 Let  $(\text{evk}, \text{pk}, \text{sk}) \leftarrow \text{mFHE.KeyGen}()$ , and denote  $\text{sk} = \mathbf{s} = (s_1, \dots, s_k)^\top$ .
3  $\text{err} := \boldsymbol{\zeta} := (\zeta^{(1)}, \dots, \zeta^{(L_{\text{Dec}})}) \leftarrow (\mathcal{R}(D_{\mathbb{Z}^n, \sigma_0 \sqrt{B_{\text{pub}}^2 \mathbf{I} - \boldsymbol{\Sigma}_{\mathbf{s}}}}))^{L_{\text{Dec}}}$ , where
    $\boldsymbol{\Sigma}_{\mathbf{s}} := \sum_{i \in [k]} \text{Gram}(s_i)$ .
4  $\{\text{sk}_i := (\mathbf{s}_i \in \mathcal{R}_q^k, \boldsymbol{\zeta}_i \in \mathcal{R}_q^{L_{\text{Dec}}})\}_{i \in [N]} \leftarrow \text{Shamir}_{\mathcal{R}_q, T, N}.\text{Share}((\text{sk}, \text{err}))$ 
5 for  $i \in [N]$  do  $\text{seed}_i \xleftarrow{\$} \{0, 1\}^\kappa$ 
6 return  $(\text{evk}, \text{pk}, \text{sk}, \{\text{sk}_i, \text{seed}_i\}_{i \in [N]})$ 
Enc( $\text{pk}, \mu \in \mathcal{M} := \mathcal{R}_2$ ):
7 return  $\text{ct} \leftarrow \text{mFHE.Enc}(\text{pk}, \mu)$  (defined in Algo. 2)
Eval( $\text{evk}, C, \text{ct}_1, \dots, \text{ct}_l$ ):
8 return  $\text{ct}_{\text{eval}} \leftarrow \text{mFHE.Eval}(\text{evk}, C, \text{ct}_1, \dots, \text{ct}_l)$  (defined in Algo. 2)
ServerDec( $\text{pk}, \text{ct} \in \mathcal{C}, \text{act}, \{\text{seed}_i\}_{i \in \text{act}}$ ):
9  $\text{sid} \leftarrow \text{sid} + 1$ , assert  $\{\text{sid} \leq L_{\text{Dec}}\}$   $\triangleright$  Internal counter ( $\text{sid} \leftarrow 0$  at setup).
10 for  $i \in \text{act}$  do  $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid}) \in \mathcal{R}_q$ 
11  $\text{ct}_{\text{fresh}} \leftarrow \text{Enc}(\text{pk}, 0)$ 
12  $\text{ct}_{\text{in}} := (\text{ct}_{\text{in}, 0}, \text{ct}_{\text{in}, 1}) := \text{ct} + \text{ct}_{\text{fresh}} \in \mathcal{R}_Q^{k+1}$ 
13  $\text{ct}_{\text{dec}, 0} := \left\lfloor \frac{1}{p} \cdot \text{ct}_{\text{in}, 0} \right\rfloor_{\sigma_0} \in \mathcal{R}_q^k$ ,  $\mathfrak{E} \leftarrow \mathcal{R}(D_{\frac{1}{2}\mathbb{Z}^n, \sigma_{\text{flood}}})$ 
14  $\text{ct}_{\text{dec}, 1} := \left\lfloor \frac{1}{p} \cdot (\text{ct}_{\text{in}, 1} + \mathfrak{E}) \right\rfloor_{\sigma_1} + \sum_{i \in \text{act}} \mathbf{m}_i \in \mathcal{R}_q$ 
15 return  $(\text{ct}_{\text{dec}} := (\text{ct}_{\text{dec}, 0}, \text{ct}_{\text{dec}, 1}) \in \mathcal{C}_{\text{dec}}, \text{sid})$ 
PartDec( $\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{b}) \in \mathcal{C}_{\text{dec}}, \text{sid}, \text{act}, \text{sk}_i := (\mathbf{s}_i, \boldsymbol{\zeta}_i), \text{seed}_i$ ):
16 assert  $\{\text{sid} \leq L_{\text{Dec}}\}$ ,  $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid}) \in \mathcal{R}_q$ 
17 return  $\text{pd}_i := \lambda_{\text{act}, i} \cdot (\bar{\mathbf{a}}^\top \mathbf{s}_i + \zeta_i^{(\text{sid})}) + \mathbf{m}_i \in \mathcal{R}_q$ 
FinDec( $\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{b}) \in \mathcal{C}_{\text{dec}}, \text{act}, \{\text{pd}_i\}_{i \in \text{act}}$ ):
18 assert  $\{|\text{act}| \geq T\}$ 
19 return  $\bar{\mu} := \lfloor (\bar{b} - \sum_{i \in \text{act}} \text{pd}_i) / \lfloor \frac{q}{2} \rfloor \rfloor$ 

```

7.1 Construction

The construction is almost identical to our LWE-based scheme. We describe our ThFHE from MLWE in Algo. 7. The threshold decryption procedure is identical to that described in Sect. 5.1.

7.2 Noise Analysis

We provide the counterpart of Lem. 6.1 in the MLWE setting, which is required for the proof of correctness and security of our MLWE-based ThFHE.

Lemma 7.1 (Noise analysis). Let χ_{mlwe} be a B_{mlwe} -bounded distribution over $\mathbb{Z}$ s.t. $\mathbf{s} := (s_1, \dots, s_k) \leftarrow \mathcal{R}(\chi_{\text{mlwe}}^n)^k$ satisfies $\min_{i \in [k]} \sigma_{\min}(\mathbf{S}_i) \geq \frac{1}{c}$ for some $c > 0$, where $\mathbf{S}_i := \text{mat}(s_i)$ (Def. 3.1). We also define $\Sigma_{\mathbf{s}} := \sum_{i \in [k]} \Sigma_{s_i}$, where $\Sigma_{s_i} := \text{Gram}(s_i)$. Let $\epsilon := \text{negl}(\kappa)$, $p \in \mathbb{N}$, $\sigma_{\text{flood}} = \Omega(2^\kappa B_{\text{eval}})$, $\sigma_{\text{flood}} \geq p\sqrt{n/2}B_{\text{mlwe}}\eta_\epsilon^+(\mathbb{Z}^n)$, $\sigma_0 \geq c\sqrt{n/2}B_{\text{mlwe}}\eta_\epsilon^+(\mathbb{Z}^n)$, $\sigma_1 \geq \eta_\epsilon(\mathbb{Z}^n)$. Let $\text{ct}_{\text{dec}} := (\text{ct}_{\text{dec},0}, \text{ct}_{\text{dec},1}) \in \mathcal{C}_{\text{dec}}$ be the output of ServerDec in Algo. 7, and define $\mathbf{e}$ as the ciphertext noise of ct_{dec} ;

$$\mathbf{e} := \text{ct}_{\text{dec},1} - \text{ct}_{\text{dec},0}^\top \mathbf{s} - \lfloor \frac{q}{2} \rfloor \cdot \mu - \sum_{i \in \text{act}} \mathbf{m}_i \in \mathcal{R}_q.$$

Then, we have

$$\mathbf{e} \approx_s \lfloor \frac{1}{p} e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}, \text{ where } e_0 \leftarrow \mathcal{R}(D_{\frac{1}{2}\mathbb{Z}^n, \sqrt{p^2 \sigma_0^2 \Sigma_{\mathbf{s}} + \sigma_{\text{flood}}^2}} \mathbf{I}). \quad (25)$$

Furthermore, for $B_{\text{pub}} > \sqrt{kn^2 B_{\text{mlwe}}^2 + 1}$ and $\zeta \leftarrow \mathcal{R}(D_{\mathbb{Z}^n, \sigma_0 \sqrt{B_{\text{pub}}^2 \mathbf{I} - \Sigma_{\mathbf{s}}}})$, we have

$$\mathbf{e} + \zeta \approx_s \lfloor \frac{1}{p} e'_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}, \text{ where } e'_0 \leftarrow \mathcal{R}(D_{\frac{1}{2}\mathbb{Z}^n, \sqrt{p^2 \sigma_0^2 B_{\text{pub}}^2 + \sigma_{\text{flood}}^2}}). \quad (26)$$

Proof. We first analyse $\text{ct}_{\text{dec}} := (\text{ct}_{\text{dec},0}, \text{ct}_{\text{dec},1})$. Let the rounding error of $\text{ct}_{\text{dec},0} := \lfloor \frac{1}{p} \cdot \text{ct}_{\text{in},0} \rfloor_{\sigma_0}$ be denoted as

$$\mathbf{r} := \text{ct}_{\text{in},0} - p \cdot \text{ct}_{\text{dec},0} \in \mathcal{R}^k. \quad (27)$$

In a manner similar to Lem. 5.1, we obtain

$$\mathbf{e} = \lfloor \frac{1}{p} (\mathbf{r}^\top \mathbf{s} - \{\frac{Q}{2}\} \cdot \mu + e_{\text{eval}} + e_{\text{fresh}} + \mathfrak{E}) + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}, \quad (28)$$

where e_{eval} and e_{fresh} are the ciphertext noises of input ciphertext ct and ct_{fresh} . We first simplify (28). By applying the smudging lemma (Lem. 3.12) coefficient-wise and under the hypothesis that $\sigma_{\text{flood}} = \Omega(2^\kappa B_{\text{eval}})$, we obtain

$$-\{\frac{Q}{2}\} \cdot \mu + e_{\text{eval}} + e_{\text{fresh}} + \mathfrak{E} \approx_s \mathfrak{E}.$$

Thus, by (28) and the above, we obtain

$$\mathbf{e} \approx_s \lfloor \frac{1}{p} (\mathbf{r}^\top \mathbf{s} + \mathfrak{E}) + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}. \quad (29)$$

Next, we analyze the distribution of $\mathbf{r}^\top \mathbf{s} + \mathfrak{E}$. For any $\mathbf{x} \in \mathcal{R}^k$, we denote the i -th elements of $\mathbf{x}$ by $x_i \in \mathcal{R}$, and j -th coefficient of x_i by $x_{i,j} \in \mathbb{Z}$. To simplify notation, we let $\mathbf{u} := \text{ct}_{\text{in},0} \in \mathcal{R}_q^k$. Since $\lfloor \cdot \rfloor_{\sigma_0}$ is performed element- and coefficient-wise, we have $r_{i,j} = u_{i,j} - p \cdot \lfloor \frac{1}{p} u_{i,j} \rfloor_{\sigma_0}$ (recall (27) and $\text{ct}_{\text{dec},0} := \lfloor \frac{1}{p} \cdot \text{ct}_{\text{in},0} \rfloor_{\sigma_0}$). Let $c_{i,j} := p \lfloor \frac{1}{p} u_{i,j} \rfloor$, and note that $c_{i,j} \in p \cdot (\frac{1}{p} \mathbb{Z}) = \mathbb{Z}$ is an integer. Then, we have $r_{i,j} \sim D_{c_{i,j} + p\mathbb{Z}, p\sigma_0}$ and $r_i = \mathcal{R}(D_{\mathbf{c}_i + p\mathbb{Z}^n, p\sigma_0})$, where $\mathbf{c}_i := (c_{i,1}, \dots, c_{i,n}) \in \mathbb{Z}^n$. Thus, we have $r_i s_i \sim \mathcal{R}(\mathbf{S}_i \cdot D_{\mathbf{c}_i + p\mathbb{Z}^n, p\sigma_0})$ by Fact 3.2. Furthermore, for any $i \in [k]$, we have

$$r_i s_i \sim \mathcal{R}(D_{A_i, p\sigma_0 \mathbf{S}_i}), \text{ where } A_i := \mathbf{S}_i \cdot \mathbf{c}_i + p\mathbf{S}_i \cdot \mathbb{Z}^n,$$

by Lem. 3.10 and the fact that $\text{mat}(a)$ is nonsingular for any $a \neq 0 \in \mathcal{R}$. Note that the coset A_i satisfies $A_i \subseteq \mathbb{Z}^n$ since the elements of $\mathbf{S}_i$ and $\mathbf{c}_i$ are integers.

We now analyse the distribution of $\mathbf{r}^\top \mathbf{s} + \mathfrak{E} = \sum_{i=1}^n r_i s_i + \mathfrak{E} \in \mathcal{R}$, where $\mathfrak{E} \leftarrow \mathcal{R}(D_{\frac{1}{2}\mathbb{Z}^n, \sigma_{\text{flood}}})$. Let $\hat{\sigma} := \min(\sigma_{\text{flood}}/p, \sigma_0/c)$, then we have $\hat{\sigma} \geq \sqrt{2n}B_{\text{mlwe}}\eta_\epsilon^+(\frac{1}{2}\mathbb{Z}^n)$ by the hypothesis. Hence, we have

$$\begin{aligned} \min(\sigma_{\text{flood}}, \min_{i \in [k]} \sigma_{\min}(p\sigma_0 \mathbf{S}_i)) &= p\hat{\sigma} \geq \sqrt{2n}pB_{\text{mlwe}}\eta_\epsilon^+(\tfrac{1}{2}\mathbb{Z}^n) \\ &\geq \sqrt{2} \max_{i \in [k]} \|p\mathbf{S}_i\|_{\text{len}} \eta_\epsilon^+(\tfrac{1}{2}\mathbb{Z}^n), \end{aligned}$$

since $\sqrt{n}B_{\text{mlwe}} \geq \max_{i \in [k]} \|\mathbf{S}_i\|_{\text{len}} = \max_{i \in [k]} \|\text{vec}(s_i)\|$ holds. Note that $\|\text{vec}(a)\| = \|\text{mat}(a)\|_{\text{len}}$ holds for any $a \in \mathcal{R}$. Therefore, by the subsequent Lem. 7.3, we obtain

$$e_0 := \mathbf{r}^\top \mathbf{s} + \mathfrak{E} \approx_s \mathcal{R}(D_{\frac{1}{2}\mathbb{Z}^n, \sqrt{\Sigma}}), \text{ where } \Sigma := p^2\sigma_0^2\Sigma_s + \sigma_{\text{flood}}^2\mathbf{I}.$$

Thus, by (29) and the above, we obtain (25).

Next, we prove (26). To simplify the notation, let $\Sigma_\zeta := B_{\text{pub}}^2\mathbf{I} - \Sigma_s$, then we have $\zeta \sim \mathcal{R}(D_{\mathbb{Z}^n, \sigma_0\sqrt{\Sigma_\zeta}})$. By Lem. 3.10, we also have $p \cdot \zeta \sim \mathcal{R}(D_{p\mathbb{Z}^n, p\sigma_0\sqrt{\Sigma_\zeta}})$. By Cor. 3.9, $e'_0 := e_0 + p \cdot \zeta \approx_s \mathcal{R}(D_{\frac{1}{2}\mathbb{Z}^n, \sqrt{\Sigma + p^2\sigma_0^2\Sigma_\zeta}}) = \mathcal{R}(D_{\frac{1}{2}\mathbb{Z}^n, \sqrt{p^2\sigma_0^2B_{\text{pub}}^2 + \sigma_{\text{flood}}^2}})$ holds if we have $\sqrt{\Sigma} \geq \eta_\epsilon(\frac{1}{2}\mathbb{Z}^n)$ and $(\Sigma^{-1} + (p^2\sigma_0^2\Sigma_\zeta)^{-1})^{-1/2} \geq \eta_\epsilon(p\mathbb{Z}^n)$. By Fact 7.2 and the hypothesis, we have

$$\sigma_{\min}(\sqrt{\Sigma}) = \sigma_{\min}(\sqrt{p^2\sigma_0^2\Sigma_s + \sigma_{\text{flood}}^2\mathbf{I}}) \geq \sqrt{2}p\hat{\sigma} \geq \sqrt{n}B_{\text{mlwe}}\eta_\epsilon^+(p\mathbb{Z}^n) \quad (30)$$

Thus, $\sqrt{\Sigma} \geq \eta_\epsilon(p\mathbb{Z}^n) > \eta_\epsilon(\frac{1}{2}\mathbb{Z}^n)$ holds by Fact 3.6. Again by Fact 7.2, we have

$$\sigma_{\min}((\Sigma^{-1} + (p^2\sigma_0^2\Sigma_\zeta)^{-1})^{-1/2}) \geq \frac{1}{\sqrt{2}} \min(\sigma_{\min}(\sqrt{\Sigma}), p\sigma_0\sigma_{\min}(\sqrt{\Sigma_\zeta})).$$

Hence, by (30) and the above, we now need only $p\sigma_0\sigma_{\min}(\sqrt{\Sigma_\zeta}) \geq \sqrt{2}\eta_\epsilon(p\mathbb{Z}^n)$, i.e., $\sigma_0\sigma_{\min}(\sqrt{\Sigma_\zeta}) \geq \sqrt{2}\eta_\epsilon(\mathbb{Z}^n)$. Since $\sigma_0 \geq \sqrt{2}\eta_\epsilon(\mathbb{Z}^n)$ by the hypothesis, we need only $\sigma_{\min}(\sqrt{\Sigma_\zeta}) \geq 1$. To prove this, it is sufficient to show $\Sigma_\zeta - \mathbf{I} = (B_{\text{pub}}^2 - 1)\mathbf{I} - \sum_{i \in [k]} \mathbf{S}_i \mathbf{S}_i^\top \succ 0$. Note that, the absolute value of every elements of $\mathbf{S}_i \mathbf{S}_i^\top$ is $\leq n\|\text{vec}(s_i)\|^2 \leq n^2B_{\text{mlwe}}^2$. Hence, $\Sigma_\zeta - \mathbf{I}$ is strictly diagonally dominant since $(B_{\text{pub}}^2 - 1) - kn^2B_{\text{mlwe}}^2 > 0$ by the hypothesis. Furthermore, this implies that $\Sigma_\zeta - \mathbf{I} \succ 0$ (c.f., [HJ85, Thm. 6.1.10]). Thus, we have $\sigma_{\min}(\sqrt{\Sigma_\zeta}) = \sqrt{\lambda_{\min}((\Sigma_\zeta - \mathbf{I}) + \mathbf{I})} \geq \sqrt{\lambda_{\min}(\Sigma_\zeta - \mathbf{I}) + 1} \geq 1$. $\square$

We complete the proof by describing the deferred Fact 7.2 and Lem. 7.3:

Fact 7.2. For any $\Sigma_1, \Sigma_2 \succ 0$, we have

$$\begin{aligned} \sigma_{\max}(\sqrt{\Sigma_1 + \Sigma_2}) &\leq \sqrt{\sigma_{\max}^2(\sqrt{\Sigma_1}) + \sigma_{\max}^2(\sqrt{\Sigma_2})} \\ \sigma_{\min}(\sqrt{\Sigma_1 + \Sigma_2}) &\geq \sqrt{2} \min\{\sigma_{\min}(\sqrt{\Sigma_1}), \sigma_{\min}(\sqrt{\Sigma_2})\} \\ \sigma_{\min}(\sqrt{(\Sigma_1^{-1} + \Sigma_2^{-1})^{-1}}) &\geq \frac{1}{\sqrt{2}} \min\{\sigma_{\min}(\sqrt{\Sigma_1}), \sigma_{\min}(\sqrt{\Sigma_2})\}. \end{aligned}$$

Lemma 7.3 (Generalized from [OT25, Lemma 5.4]). *Let $\epsilon = \text{negl}(\kappa)$. Let $\Sigma_0, \dots, \Sigma_k \in \mathbb{R}^{n \times n}$ be positive definite matrices. Let $A_0, \dots, A_k$ be cosets of full-rank lattices $\mathcal{L}_0(\mathbf{B}_0), \dots, \mathcal{L}_k(\mathbf{B}_k)$ with (nonsingular) basis $\mathbf{B}_0, \dots, \mathbf{B}_k$ s.t. $A_1, \dots, A_k \subseteq A_0$. Let $\hat{\sigma} := \min_{i \in \{0,1,\dots,k\}} \sigma_{\min}(\sqrt{\Sigma_i})$ and $\hat{B} := \max_{i \in [k]} \|\mathbf{B}_i\|_{\text{len}}$. If $\hat{\sigma} \geq \sqrt{2}\hat{B}\|\mathbf{B}_0\|_{\text{len}}\eta_\epsilon^+(\mathbb{Z}^n)$, we have*

$$\sum_{i=1}^k D_{A_i, \sqrt{\Sigma_i}} + D_{A_0, \sqrt{\Sigma_0}} \approx_s D_{A_0, \sqrt{\sum_{i=0}^k \Sigma_i}}. \quad (31)$$

Proof. The proof is almost identical to that of [OT25, Lemma 5.4]. For completeness, we provide the proof in Appendix C. $\square$

7.3 Correctness

Since the syntax is identical, the correctness of our MLWE-based ThFHE (Algo. 7) is also defined as in Def. 5.2. We prove the correctness:

Theorem 7.4 (Correctness). *Assume that parameters $\epsilon, p, \chi_{\text{mlwe}}, B_{\text{mlwe}}, B_{\text{pub}}, \sigma_{\text{flood}}, \sigma_0, \sigma_1$ are selected as in Lem. 7.1. Furthermore, assume $q = \sigma_{\text{dec}} \cdot \Omega(\sqrt{\kappa})$, where $\sigma_{\text{dec}} := \sqrt{(\sigma_0 B_{\text{pub}})^2 + (\sigma_{\text{flood}}/p)^2} + \sigma_1$. Then, mThFHE (Algo. 7) is correct if the underlying mFHE (Algo. 2) is correct.*

Proof. The proof is essentially identical to Thm. 6.2, except for that we rely on Lem. 7.1 instead of Lem. 6.1. $\square$

7.4 Security

The security of our mThFHE (Algo. 7) is almost identically defined as Def. 5.4, by replacing variables over $\mathbb{Z}_q^n$ with the corresponding variables over R_q^k . The security can be proven by MLWE assumption as follows:

Theorem 7.5 (Security). *mThFHE (Algo. 7) is simulation-secure under the assumption on the pseudorandomness of PRF and MLWE assumption: For any PPT algorithms $\mathcal{D}, \mathcal{A}$, there exists PPT algorithms $\text{Sim}, \mathcal{B}_{\text{LWE}}$ and $\mathcal{B}_{\text{PRF}}$ s.t.*

$$\text{Adv}_{\mathcal{D}, \mathcal{A}}^{\text{ThFHE}}(\kappa) < \text{Adv}_{\mathcal{B}_{\text{LWE}}}^{\text{MLWE}(m,k,Q,\mathcal{R}(\chi_{\text{mlwe}}^n))} + \text{Adv}_{\mathcal{B}_{\text{PRF}}}^{\text{PRF}}(\kappa) + \text{negl}(\kappa),$$

if $\epsilon, p, q, Q, B_{\text{mlwe}}, B_{\text{pub}}, \chi_{\text{mlwe}}, \sigma_{\text{flood}}, \sigma_0, \sigma_1$ are selected as in Thm. 7.4.

Proof. The proof is almost identical to that of Thm. 5.5, except for that we rely on Lem. 7.1 instead of Lem. 6.1. $\square$

Acknowledgement. This work was supported by JST K Program Grant Number JPMJKP24U2, Japan.

References

- [ABV⁺12] S. Agrawal, X. Boyen, V. Vaikuntanathan, P. Voulgaris, H. Wee. “Functional Encryption for Threshold Functions (or Fuzzy IBE) from Lattices”. *PKC 2012*. 2012, pp. 280–297. https://doi.org/10.1007/978-3-642-30057-8_17.
- [ARS⁺15] M. R. Albrecht, C. Rechberger, T. Schneider, T. Tiessen, M. Zohner. “Ciphers for MPC and FHE”. *EUROCRYPT 2015*. 2015, pp. 430–454. https://doi.org/10.1007/978-3-662-46800-5_17.
- [AJL⁺12] G. Asharov, A. Jain, A. López-Alt, E. Tromer, V. Vaikuntanathan, D. Wichs. “Multiparty Computation with Low Communication, Computation and Interaction via Threshold FHE”. *EUROCRYPT 2012*. 2012, pp. 483–501. https://doi.org/10.1007/978-3-642-29011-4_29.
- [BJMS20] S. Badrinarayanan, A. Jain, N. Manohar, A. Sahai. “Secure MPC: Laziness Leads to GOD”. *ASIACRYPT 2020*. 2020, pp. 120–150. https://doi.org/10.1007/978-3-030-64840-4_5.
- [BCK⁺23] Y. Bae, J. H. Cheon, J. Kim, J. H. Park, D. Stehlé. “HERMES: Efficient Ring Packing Using MLWE Ciphertexts and Application to Transciphering”. *CRYPTO 2023*. 2023, pp. 37–69. https://doi.org/10.1007/978-3-031-38551-3_2.
- [BGL⁺23] J. Bell, A. Gascón, T. Lepoint, B. Li, S. Meiklejohn, M. Raykova, C. Yun. “ACORN: Input Validation for Secure Aggregation”. *USENIX Security 23*. 2023, pp. 4805–4822.
- [BBG⁺20] J. H. Bell, K. A. Bonawitz, A. Gascón, T. Lepoint, M. Raykova. “Secure Single-Server Aggregation with (Poly)Logarithmic Overhead”. *ACM CCS '20*. 2020, pp. 1253–1269. <https://doi.org/10.1145/3372297.3417885>.
- [BD10] R. Bendlin, I. Damgård. “Threshold Decryption and Zero-Knowledge Proofs for Lattice-Based Cryptosystems”. *TCC 2010*. 2010, pp. 201–218. https://doi.org/10.1007/978-3-642-11799-2_13.
- [BGG⁺18] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, A. Sahai. “Threshold Cryptosystems from Threshold Fully Homomorphic Encryption”. *CRYPTO 2018*. 2018, pp. 565–596. https://doi.org/10.1007/978-3-319-96884-1_19.
- [BS23] K. Boudgoust, P. Scholl. “Simple Threshold (Fully Homomorphic) Encryption from LWE with Polynomial Modulus”. *ASIACRYPT 2023*. 2023, pp. 371–404. https://doi.org/10.1007/978-981-99-8721-4_12.
- [BS24] K. Boudgoust, P. Scholl. *Simple Threshold (Fully Homomorphic) Encryption From LWE With Polynomial Modulus*. ePrint 2023/016. 2024. <https://eprint.iacr.org/2023/016>. Version 2024-07-16.

- [BGV12] Z. Brakerski, C. Gentry, V. Vaikuntanathan. “(Leveled) Fully Homomorphic Encryption without Bootstrapping”. *ITCS 2012*. ACM, 2012, pp. 309–325. <https://doi.org/10.1145/2090236.2090262>.
- [BP23] L. T. Brandão, R. Peralta. *NIST IR 8214C ipd: NIST First Call for Multi-Party Threshold Schemes (Initial Public Draft)*. 2023. <https://doi.org/10.6028/NIST.IR.8214C.ipd>.
- [CHK⁺18] J. H. Cheon, K. Han, A. Kim, M. Kim, Y. Song. “Bootstrapping for Approximate Homomorphic Encryption”. *EUROCRYPT 2018*. 2018, pp. 360–384. https://doi.org/10.1007/978-3-319-78381-9_14.
- [CKKS17] J. H. Cheon, A. Kim, M. Kim, Y. Song. “Homomorphic Encryption for Arithmetic of Approximate Numbers”. *ASIACRYPT 2017*. 2017, pp. 409–437. https://doi.org/10.1007/978-3-319-70694-8_15.
- [CGGI20] I. Chillotti, N. Gama, M. Georgieva, M. Izabachène. “TFHE: Fast Fully Homomorphic Encryption Over the Torus”. *J. Cryptol.* 33.1 (2020), pp. 34–91. <https://doi.org/10.1007/s00145-019-09319-x>.
- [DEG⁺18] C. Dobraunig, M. Eichlseder, L. Grassi, V. Lallemand, G. Leander, E. List, F. Mendel, C. Rechberger. “Rasta: A Cipher with Low ANDdepth and Few ANDs per Bit”. *CRYPTO 2018*. 2018, pp. 662–692. https://doi.org/10.1007/978-3-319-96884-1_22.
- [GGHR14] S. Garg, C. Gentry, S. Halevi, M. Raykova. “Two-Round Secure MPC from Indistinguishability Obfuscation”. *TCC 2014*. 2014, pp. 74–94. https://doi.org/10.1007/978-3-642-54242-8_4.
- [GMPW20] N. Genise, D. Micciancio, C. Peikert, M. Walter. “Improved Discrete Gaussian and SubGaussian Analysis for Lattice Cryptography”. *PKC 2020*. 2020, pp. 623–651. https://doi.org/10.1007/978-3-030-45374-9_21.
- [Gen09] C. Gentry. “Fully Homomorphic Encryption Using Ideal Lattices”. *STOC '09*. ACM, 2009, pp. 169–178. <https://doi.org/10.1145/1536414.1536440>.
- [GPV08] C. Gentry, C. Peikert, V. Vaikuntanathan. “Trapdoors for Hard Lattices and New Cryptographic Constructions”. *STOC '08*. ACM, 2008, pp. 197–206. <https://doi.org/10.1145/1374376.1374407>.
- [GSW13] C. Gentry, A. Sahai, B. Waters. “Homomorphic Encryption from Learning with Errors: Conceptually-Simpler, Asymptotically-Faster, Attribute-Based”. *CRYPTO 2013*. 2013, pp. 75–92. https://doi.org/10.1007/978-3-642-40041-4_5.
- [Gol20] O. Goldreich. “On (Valiant’s) Polynomial-Size Monotone Formula for Majority”. *Computational Complexity and Property Testing: On the Interplay Between Randomness and Computation*. 2020, pp. 17–23. https://doi.org/10.1007/978-3-030-43662-9_3.
- [GLS15] S. Dov Gordon, F.-H. Liu, E. Shi. “Constant-Round MPC with Fairness and Guarantee of Output Delivery”. *CRYPTO 2015*. 2015, pp. 63–82. https://doi.org/10.1007/978-3-662-48000-7_4.

- [HKL⁺22] J. Ha, S. Kim, B. Lee, J. Lee, M. Son. “Rubato: Noisy Ciphers for Approximate Homomorphic Encryption”. *EUROCRYPT 2022*. 2022, pp. 581–610. https://doi.org/10.1007/978-3-031-06944-4_20.
- [HJ85] R. A. Horn, C. R. Johnson. *Matrix Analysis*. Cambridge University Press, 1985. <https://doi.org/10.1017/CBO9780511810817>.
- [LP11] R. Lindner, C. Peikert. “Better Key Sizes (and Attacks) for LWE-Based Encryption”. *CT-RSA 2011*. 2011, pp. 319–339. https://doi.org/10.1007/978-3-642-19074-2_21.
- [LPR10] V. Lyubashevsky, C. Peikert, O. Regev. “On Ideal Lattices and Learning with Errors over Rings”. *EUROCRYPT 2010*. 2010, pp. 1–23. https://doi.org/10.1007/978-3-642-13190-5_1.
- [MP13] D. Micciancio, C. Peikert. “Hardness of SIS and LWE with Small Parameters”. *CRYPTO 2013*. 2013, pp. 21–39. https://doi.org/10.1007/978-3-642-40041-4_2.
- [MR07] D. Micciancio, O. Regev. “Worst-Case to Average-Case Reductions Based on Gaussian Measures”. *SIAM J. Comput.* 37.1 (2007), pp. 267–302. <https://doi.org/10.1137/S0097539705447360>.
- [MS25] D. Micciancio, A. Suhl. “Simulation-Secure Threshold PKE from LWE with Polynomial Modulus”. *IACR Communications in Cryptology* 1.4 (Jan. 13, 2025). <https://doi.org/10.62056/a0zogy4e>.
- [NLV11] M. Naehrig, K. Lauter, V. Vaikuntanathan. “Can homomorphic encryption be practical?” *CCSW ’11*. ACM, 2011, pp. 113–124. <https://doi.org/10.1145/2046660.2046682>.
- [OT25] H. Okada, T. Takagi. “Gram Root Decomposition over the Polynomial Ring: Application to Sphericalization of Discrete Gaussian”. *ICISSP 2025*. 2025, pp. 306–317. <https://doi.org/10.5220/0013139700003899>.
- [OT26] H. Okada, T. Takagi. “Low Communication Threshold FHE from Standard (Module-)LWE”. *ASIACRYPT 2025*. Springer Nature Singapore, 2026, pp. 165–198. https://doi.org/10.1007/978-981-95-5122-4_6.
- [PS25] A. Passelègue, D. Stehlé. “Low Communication Threshold Fully Homomorphic Encryption”. *ASIACRYPT 2024*. 2025, pp. 297–329. https://doi.org/10.1007/978-981-96-0875-1_10.
- [Pei10] C. Peikert. “An Efficient and Parallel Gaussian Sampler for Lattices”. *CRYPTO 2010*. 2010, pp. 80–97. https://doi.org/10.1007/978-3-642-14623-7_5.
- [PKM⁺24] R. del Pino, S. Katsumata, M. Maller, F. Mouhartem, T. Prest, M.-J. Saarinen. “Threshold Raccoon: Practical Threshold Signatures from Standard Lattice Assumptions”. *EUROCRYPT 2024*. 2024, pp. 219–248. https://doi.org/10.1007/978-3-031-58723-8_8.
- [Reg09] O. Regev. “On Lattices, Learning with Errors, Random Linear Codes, and Cryptography”. *J. ACM* 56.6 (2009). <https://doi.org/10.1145/1591262.1591263>.

- [org/10.1145/1568318.1568324](https://doi.org/10.1145/1568318.1568324). Preliminary version appeared in STOC '05.
- [Sha79] A. Shamir. “How to share a secret”. *Commun. ACM* 22.11 (1979), pp. 612–613. <https://doi.org/10.1145/359168.359176>.
- [Val84] L. Valiant. “Short monotone formulae for the majority function”. *Journal of Algorithms* 5.3 (1984), pp. 363–366. [https://doi.org/10.1016/0196-6774\(84\)90016-6](https://doi.org/10.1016/0196-6774(84)90016-6).

A Definition of Pseudorandom Function

In this section, we provide the (standard) definitions of pseudorandom function which is deferred from Sect. 3. We rely on the multi-instance variant of the pseudorandom functions PRF given in [PKM⁺24]. Here, we refer to the definition for the completeness.

Definition A.1 (Pseudorandom function). *Let κ be a security parameter and $n, l \in \mathbb{N}$. We say that a deterministic PPT algorithm $\text{PRF} : \{0, 1\}^\kappa \times \{0, 1\}^n \rightarrow \{0, 1\}^l$ is pseudorandom function if for any PPT algorithm $\mathcal{A}$*

$$\text{Adv}_{\mathcal{A}}^{\text{PRF}}(\kappa) := |\Pr[\text{Game}_{\mathcal{A}}^{\text{PRF}}(\kappa) = 1] - \frac{1}{2}| = \text{negl}(\kappa)$$

holds, where $\text{Game}_{\mathcal{A}}^{\text{PRF}}$ is defined as in Algo. 8.

Algorithm 8: Security game of PRF (Def. A.1).

$\text{Game}_{\mathcal{A}}^{\text{PRF}}(\kappa)$:	$\text{O}_{\text{PRF}}(i, x \in \{0, 1\}^n)$:
1 $\text{List}[\cdot] := \emptyset$	6 if $\text{List}[i] = \emptyset$ then
2 $b \leftarrow \{0, 1\}$	$\text{seed} \leftarrow \{0, 1\}^\kappa$, $\text{List}[i] \leftarrow \text{seed}$
3 $b' \leftarrow \mathcal{A}^{\text{O}_{\text{PRF}}}(\kappa)$	7 $\text{seed} \leftarrow \text{List}[i]$
4 if $b = b'$ then return 1	8 $y_0 \leftarrow \mathcal{U}(\{0, 1\}^l)$, $y_1 \leftarrow \text{PRF}(\text{seed}, x)$
5 else return 0	9 return y_b

B Full Proof of Theorem 5.5

We provide the full formal proof of Thm. 5.5; the claim holds by the subsequent Lemmas B.1–B.7. We define hybrid experiments $\text{Hyb}_1, \dots, \text{Hyb}_6$ in Algorithms 9, ..., 14, respectively. We now prove the deferred lemmas, i.e., the indistinguishability of the hybrid experiments. As a shorthand, for any algorithm $\mathcal{D}$ and experiments $\text{H}_0(1^\kappa)$ and $\text{H}_1(1^\kappa)$, we define $\text{Adv}_{\mathcal{D}}^{\text{H}_0-\text{H}_1}(\kappa) := |\Pr[\mathcal{D}(\text{H}_0(1^\kappa)) = 1] - \Pr[\mathcal{D}(\text{H}_1(1^\kappa)) = 1]|$.

Lemma B.1 ($\text{Expt}_{\mathcal{A}, \text{Real}} \approx_s \text{Hyb}_1$). *Assume $\epsilon, p, q, Q, \chi_{\text{lwe}}, B_{\text{lwe}}, B_{\text{pub}}, \sigma_{\text{flood}}, \sigma_0, \sigma_1$ are selected as in Thm. 5.3. Then, we have $\text{Adv}_{\mathcal{D}}^{\text{Hyb}_1-\text{Hyb}_2}(\kappa) = \text{negl}(\kappa)$.*

Proof. The claim holds by Thm. 5.3. □

Algorithm 9: Hyb₁: Differences from Expt_{A,Real} are highlighted.

▷ Expt, OEnc, OEval and ODec are identical to those in Expt_{A,Real}
ServerDec(pk, ct, act, {seed_i}_{i∈act}):
 ▷ Perform lines 9–12 of ServerDec in Algo. 3
 1 $\text{ct}_{\text{dec},0} := \left\lfloor \frac{1}{p} \cdot \text{ct}_{\text{in},0} \right\rfloor_{\sigma_0} \bmod q$
 2 $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}, \mathbf{e} := \left\lfloor \frac{1}{p}e_0 + \left\{\frac{q}{2}\right\} \cdot \mu \right\rfloor_{\sigma_1}$
 3 $\text{ct}_{\text{dec},1} = \text{ct}_{\text{dec},0}^\top \mathbf{s} + \mathbf{e} + \left\lfloor \frac{q}{2} \right\rfloor \cdot \mu + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$, where μ is the plaintext of ct
 4 **return** (ct_{dec} := (ct_{dec,0}, ct_{dec,1}) ∈ C_{dec}, sid)

Algorithm 10: Hyb₂: Differences from Hyb₁ are highlighted.

▷ Expt, OEnc, OEval and ODec are identical to those in Expt_{A,Real}
ServerDec(pk, ct, act, {seed_i}_{i∈act}):
 1 $\text{sid} \leftarrow \text{sid} + 1$, **assert** {sid ≤ L_{Dec}}
 2 **for** $i \in \text{act}$ **do** $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid}) \in \mathbb{Z}_q$
 3 $\text{ct}_{\text{dec},0} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$
 4 $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}, \mathbf{e} := \left\lfloor \frac{1}{p}e_0 + \left\{\frac{q}{2}\right\} \cdot \mu \right\rfloor_{\sigma_1}$
 5 $\text{ct}_{\text{dec},1} := \text{ct}_{\text{dec},0}^\top \mathbf{s} + \mathbf{e} + \left\lfloor \frac{q}{2} \right\rfloor \cdot \mu + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$, where μ is the plaintext of ct
 6 **return** (ct_{dec} := (ct_{dec,0}, ct_{dec,1}) ∈ C_{dec}, sid)

Lemma B.2 (Hyb₁ ≈_c Hyb₂). $\text{Adv}_{\mathcal{D}}^{\text{Hyb}_1 - \text{Hyb}_2}(\kappa) \leq \text{Adv}_{\mathcal{B}_{\text{LWE}}}^{\text{LWE}(m,n,Q,\chi_{\text{lwe}})}(\kappa)$.

Proof. Let $\text{ct}_{\text{fresh}} := (\mathbf{a}, b) \leftarrow \text{Enc}_{\text{pk}}(0)$ in the original ServerDec. Recall $\mathbf{a} := \mathbf{r}^\top \mathbf{A} + \mathbf{f}$ as defined in Algo. 1, then we have $\mathbf{a} \approx_c \mathcal{U}(\mathbb{Z}_q^n)$ under the LWE assumption. Since ct_{fresh} is not used elsewhere, we have $\text{ct}_{\text{in},0} \approx_c \mathcal{U}(\mathbb{Z}_q^n)$. Similarly, $\text{ct}_{\text{dec},0} := \left\lfloor \frac{1}{p} \cdot \text{ct}_{\text{in},0} \right\rfloor_{\sigma_0} \bmod q \approx_c \mathcal{U}(\mathbb{Z}_q^n)$ also holds. □

Lemma B.3 (Hyb₂ ≈_c Hyb₃). We have $\text{Adv}_{\mathcal{D}}^{\text{Hyb}_2 - \text{Hyb}_3} \leq \text{Adv}_{\mathcal{B}_{\text{PRF}}}^{\text{PRF}}(\kappa)$.

Proof. In Hyb₃, the challenger samples $\mathbf{m}_i \leftarrow \mathcal{U}(\mathbb{Z}_q)$ for all $i \in \text{hon}_{\text{act}}$ instead of the output of PRF. It is easy to see that this hybrid is indistinguishable from Hyb₂ under the assumption on the pseudorandomness of the PRF (Def. A.1). □

Lemma B.4 (Hyb₃ ≈ Hyb₄). The distributions of Hyb₃ and Hyb₄ are identical.

Proof. We analyse the distribution of the partial decryption of the party P_h in Hyb₄; $\text{pd}_h := \lambda_{\text{act},h} \cdot \bar{\mathbf{a}}^\top \mathbf{s}_h + \mathbf{m}_h$. By the correctness of Shamir secret sharing, we have $\sum_{i \in \text{act}} \text{pd}_i = \bar{\mathbf{a}}^\top \mathbf{s} + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$. Furthermore, the output $(\bar{\mathbf{a}}, \bar{b})$ of ServerDec (in both Hyb₄ and Hyb₅) satisfies $\bar{b} := \bar{\mathbf{a}}^\top \mathbf{s} + \mathbf{e} + \left\lfloor \frac{q}{2} \right\rfloor \cdot \mu + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$. Hence, we obtain $\sum_{i \in \text{act}} \text{pd}_i = \bar{b} - \left\lfloor \frac{q}{2} \right\rfloor \cdot \mu - \mathbf{e}$, which is equivalent to $\text{pd}_h = \bar{b} - \left\lfloor \frac{q}{2} \right\rfloor \cdot \mu - \sum_{i \in \text{act} \setminus \{h\}} \text{pd}_i - \mathbf{e}$. □

Algorithm 11: Hyb₃: Differences from Hyb₂ are highlighted.

▷ Expt, OEnc and OEval are identical to those in $\text{Expt}_{\mathcal{A}, \text{Real}}$

ODec(act):

▷ Perform lines 9–10 of $\text{Expt}_{\mathcal{A}, \text{Real}}.\text{ODec}(\text{act})$ in Algo. 5

- 1 $\text{sid} \leftarrow \text{sid} + 1$, **assert** $\{\text{sid} \leq L_{\text{Dec}}\}$
- 2 **for** $i \in \text{corr}_{\text{act}}$ **do** $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid})$
- 3 **for** $i \in \text{hon}_{\text{act}}$ **do** $\mathbf{m}_i \leftarrow \mathcal{U}(\mathbb{Z}_q)$ ▷ Sample uniformly
- 4 $\bar{\mathbf{a}} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$, $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}$, $\mathbf{e} := \lfloor \frac{1}{p}e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$
- 5 $\bar{b} := \bar{\mathbf{a}}^\top \mathbf{s} + \mathbf{e} + \lfloor \frac{q}{2} \rfloor \cdot \mu + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$, where μ is the plaintext of ct
- 6 **for** $i \in \text{act}$ **do** $\text{pd}_i := \lambda_{\text{act}, i} \cdot \bar{\mathbf{a}}^\top \mathbf{s}_i + \mathbf{m}_i$
- 7 **return** $(\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{b}), \{\text{pd}_i\}_{i \in \text{act}})$

Algorithm 12: Hyb₄: Differences from Hyb₃ are highlighted.

▷ Expt, OEnc and OEval are identical to those in $\text{Expt}_{\mathcal{A}, \text{Real}}$

ODec(act):

▷ Perform lines 9–10 of $\text{Expt}_{\mathcal{A}, \text{Real}}.\text{ODec}(\text{act})$ in Algo. 5

- 1 $\text{sid} \leftarrow \text{sid} + 1$, **assert** $\{\text{sid} \leq L_{\text{Dec}}\}$
- 2 **for** $i \in \text{corr}_{\text{act}}$ **do** $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid})$
- 3 **for** $i \in \text{hon}_{\text{act}}$ **do** $\mathbf{m}_i \leftarrow \mathcal{U}(\mathbb{Z}_q)$
- 4 $\bar{\mathbf{a}} \leftarrow \mathcal{U}(\mathbb{Z}_q^n)$, $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0\|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}$, $\mathbf{e} := \lfloor \frac{1}{p}e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$
- 5 $\bar{b} := \bar{\mathbf{a}}^\top \mathbf{s} + \mathbf{e} + \lfloor \frac{q}{2} \rfloor \cdot \mu + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$, where μ is the plaintext of ct
- 6 Fix some $h \in \text{hon}_{\text{act}}$
- 7 **for** $i \in \text{act} \setminus \{h\}$ **do** $\text{pd}_i := \lambda_{\text{act}, i} \cdot \bar{\mathbf{a}}^\top \mathbf{s}_i + \mathbf{m}_i$
- 8 $\text{pd}_h := \bar{b} - \lfloor \frac{q}{2} \rfloor \cdot \mu - \sum_{i \in \text{act} \setminus \{h\}} \text{pd}_i - \mathbf{e}$
- 9 **return** $(\text{ct}_{\text{dec}} := (\bar{\mathbf{a}}, \bar{b}), \{\text{pd}_i\}_{i \in \text{act}})$

Lemma B.5 (Hyb₄ $\approx$ Hyb₅). *The distributions of Hyb₄ and Hyb₅ are identical.*

Proof. In Hyb₄, $\mathbf{m}_h \leftarrow \mathcal{U}(\mathbb{Z}_q)$ is used only for calculating $\bar{b} := \bar{\mathbf{a}}^\top \mathbf{s} + \mathbf{e} + \lfloor \frac{q}{2} \rfloor \cdot \mu + \sum_{i \in \text{act}} \mathbf{m}_i \bmod q$ since pd_h in Hyb₄ no longer uses $\mathbf{m}_h$. Thus, we have $\bar{b} \sim \mathcal{U}(\mathbb{Z}_q)$, and it is identically distributed as $\bar{b}$ in Hyb₅. $\square$

Lemma B.6 (Hyb₅ $\approx$ Hyb₆). *The distributions of Hyb₅ and Hyb₆ are identical.*

Proof. In Hyb₅, $\mathbf{m}_i \leftarrow \mathcal{U}(\mathbb{Z}_q)$ for $i \in \text{hon}_{\text{act}} \setminus \{h\}$ are used only for calculating pd_i . Hence, we have $\text{pd}_i \sim \mathcal{U}(\mathbb{Z}_q)$, which are identically distributed as in Hyb₆. $\square$

Lemma B.7 (Hyb₆ $\approx$ $\text{Expt}_{\mathcal{A}, \text{Ideal}}$). *The distributions of Hyb₆ and $\text{Expt}_{\mathcal{A}, \text{Ideal}}$ are identical.*

Proof. Let the PPT simulator Sim of $\text{Expt}_{\mathcal{A}, \text{Ideal}}$ in Algo. 5 as the procedures of lines 2–9 of Hyb₆. Note that $\mathbf{e} \leftarrow D_{\mathbb{Z}, \sqrt{(\sigma_0\|\mathbf{s}\|)^2 + (\sigma_{\text{flood}}/p)^2 + \sigma_1^2}}$ in line 8 is simulatable since pk contains $\|\mathbf{s}\|$ (see line 3 in Algo. 3). The only difference between

Algorithm 13: Hyb₅: Differences from Hyb₄ are highlighted.

$\triangleright$ Expt, OEnc and OEval are identical to those in $\text{Expt}_{\mathcal{A}, \text{Real}}$
ODec(act):
 $\triangleright$ Perform lines 9–10 of $\text{Expt}_{\mathcal{A}, \text{Real}} \cdot \text{ODec}(\text{act})$ in Algo. 5
1 **sid** $\leftarrow$ **sid** + 1, **assert** {**sid** $\leq$ L_{Dec} }
2 **for** $i \in \text{corr}_{\text{act}}$ **do** $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid})$
3 Fix some $h \in \text{hon}_{\text{act}}$
4 **for** $i \in \text{hon}_{\text{act}} \setminus \{h\}$ **do** $\mathbf{m}_i \leftarrow \mathcal{U}(\mathbb{Z}_q)$ $\triangleright \mathbf{m}_h$ is no longer sampled
5 $\bar{\mathbf{a}} \leftarrow \mathcal{U}(\mathbb{Z}_q^n), \bar{b} \leftarrow \mathcal{U}(\mathbb{Z}_q)$ $\triangleright$ Sample uniformly
6 **for** $i \in \text{act} \setminus \{h\}$ **do** $\text{pd}_i := \lambda_{\text{act}, i} \cdot \bar{\mathbf{a}}^\top \mathbf{s}_i + \mathbf{m}_i$
7 $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0 \|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}, \mathbf{e} := \lfloor \frac{1}{p} e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$
8 $\text{pd}_h := \bar{b} - \lfloor \frac{q}{2} \rfloor \cdot \mu - \sum_{i \in \text{act} \setminus \{h\}} \text{pd}_i - \mathbf{e}$
9 **return** (**ct**_{dec} := $(\bar{\mathbf{a}}, \bar{b}), \{\text{pd}_i\}_{i \in \text{act}}$)

Algorithm 14: Hyb₆: Differences from Hyb₅ are highlighted.

$\triangleright$ Expt, OEnc and OEval are identical to those in $\text{Expt}_{\mathcal{A}, \text{Real}}$
ODec(act):
 $\triangleright$ Perform lines 9–10 of $\text{Expt}_{\mathcal{A}, \text{Real}} \cdot \text{ODec}(\text{act})$ in Algo. 5
1 **sid** $\leftarrow$ **sid** + 1, **assert** {**sid** $\leq$ L_{Dec} }
2 **for** $i \in \text{corr}_{\text{act}}$ **do** $\mathbf{m}_i := \text{PRF}(\text{seed}_i, \text{sid})$
3 Fix some $h \in \text{hon}_{\text{act}}$ $\triangleright \mathbf{m}_i$ is no longer sampled for $i \in \text{hon}_{\text{act}}$
4 $(\bar{\mathbf{a}}, \bar{b}) \leftarrow \mathcal{U}(\mathbb{Z}_q^n \times \mathbb{Z}_q)$
5 **for** $i \in \text{corr}_{\text{act}}$ **do** $\text{pd}_i := \lambda_{\text{act}, i} \cdot \bar{\mathbf{a}}^\top \mathbf{s}_i + \mathbf{m}_i$
6 **for** $i \in \text{hon}_{\text{act}} \setminus \{h\}$ **do** $\text{pd}_i \leftarrow \mathcal{U}(\mathbb{Z}_q)$ $\triangleright$ Sample uniformly
7 $e_0 \leftarrow D_{\frac{1}{2}\mathbb{Z}, \sqrt{(p\sigma_0 \|\mathbf{s}\|)^2 + \sigma_{\text{flood}}^2}}, \mathbf{e} := \lfloor \frac{1}{p} e_0 + \{\frac{q}{2}\} \cdot \mu \rfloor_{\sigma_1}$
8 $\text{pd}_h := \bar{b} - \lfloor \frac{q}{2} \rfloor \cdot \mu - \sum_{i \in \text{act} \setminus \{h\}} \text{pd}_i - \mathbf{e}$
9 **return** (**ct**_{dec} := $(\bar{\mathbf{a}}, \bar{b}), \{\text{pd}_i\}_{i \in \text{act}}$)

Hyb₆ and $\text{Expt}_{\mathcal{A}, \text{Ideal}}$ is that the secret shares $\{\mathbf{sk}_i\}_{i \in [N]} \leftarrow \text{Shamir.Share}_{\mathbb{Z}_q, T, N}(\mathbf{sk})$ in Hyb₆ are replaced with $\{\mathbf{sk}_i\}_{i \in [N]} \leftarrow \text{Shamir.Share}_{\mathbb{Z}_q, T, N}(\mathbf{0})$ in $\text{Expt}_{\mathcal{A}, \text{Ideal}}$, which contain no information about $\mathbf{sk}$. In Hyb₆ and $\text{Expt}_{\mathcal{A}, \text{Ideal}}$, only the corrupted secret shares $\{\mathbf{sk}_i\}_{i \in \text{corr}_{\text{act}}}$ are used, and $|\text{corr}_{\text{act}}| \leq |\text{corr}| < T$ holds for any sid . Hence, Hyb₆ and $\text{Expt}_{\mathcal{A}, \text{Ideal}}$ are identically distributed owing to the privacy of Shamir secret sharing (Constr. 3.17). $\square$

C Proof of Lemma 7.3

Although the proof of Lem. 7.3 is almost identical to that of [OT25, Lemma 5.4], we provide the proof for the completeness:

Proof. By using Cor. 3.9, we first show

$$D_{A_1, \sqrt{\Sigma_1}} + D_{A_0, \sqrt{\Sigma_0}} \approx_s D_{\mathbb{Z}^n, \sqrt{\Sigma_0 + \Sigma_1}}. \quad (32)$$

Note that we have $\hat{\sigma} \geq \sqrt{2}\hat{B}\|\mathbf{B}_0\|_{\text{len}}\eta_\epsilon^+(\mathbb{Z}^n) \geq \|\mathbf{B}_0\|_{\text{len}}\eta_\epsilon^+(\mathbb{Z}^n) \geq \eta_\epsilon(A_0)$ by the hypothesis (and Fact 3.6). Hence, we also obtain $\sqrt{\boldsymbol{\Sigma}_0} \geq \eta_\epsilon(A_0)$ since $\sigma_{\min}(\sqrt{\boldsymbol{\Sigma}_0}) \geq \hat{\sigma}$. In addition, we have $\sqrt{(\boldsymbol{\Sigma}_0^{-1} + \boldsymbol{\Sigma}_1^{-1})^{-1}} \geq \eta_\epsilon(\mathcal{L}_1(\mathbf{B}_1))$ because

$$\begin{aligned} \sigma_{\min}(\sqrt{(\boldsymbol{\Sigma}_0^{-1} + \boldsymbol{\Sigma}_1^{-1})^{-1}}) &\geq \frac{1}{\sqrt{2}} \min\{\sigma_{\min}(\sqrt{\boldsymbol{\Sigma}_0}), \sigma_{\min}(\sqrt{\boldsymbol{\Sigma}_1})\} \\ &\geq \frac{1}{\sqrt{2}} \hat{\sigma} \geq \|\mathbf{B}_1\|_{\text{len}}\eta_\epsilon^+(\mathbb{Z}^n) \end{aligned}$$

by Fact 7.2 and the hypothesis $\hat{\sigma} \geq \sqrt{2}\hat{B}\eta_\epsilon^+(\mathbb{Z}^n) \geq \sqrt{2}\|\mathbf{B}_1\|_{\text{len}}\eta_\epsilon^+(\mathbb{Z}^n)$. Therefore, we obtain (32) by Cor. 3.9 since $A_1 \subseteq A_0$. Next, we show

$$D_{A_2, \sqrt{\boldsymbol{\Sigma}_2}} + D_{A_0, \sqrt{\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1}} \approx_s D_{A_0, \sqrt{\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2}} \quad (33)$$

via Cor. 3.9 again. By Fact 3.6, Fact 7.2 and $\hat{\sigma} \geq \eta_\epsilon(A_0)$, we have $\sqrt{\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1} \geq \eta_\epsilon(A_0)$ because we have

$$\sigma_{\min}(\sqrt{\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1}) \geq \min\{\sigma_{\min}(\sqrt{\boldsymbol{\Sigma}_0}), \sigma_{\min}(\sqrt{\boldsymbol{\Sigma}_1})\} \geq \hat{\sigma} \geq \eta_\epsilon(A_0).$$

Furthermore, we have $\sqrt{((\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1)^{-1} + \boldsymbol{\Sigma}_2^{-1})^{-1}} \geq \eta_\epsilon(\mathcal{L}_1(\mathbf{B}_2))$ because

$$\begin{aligned} \sigma_{\min}(\sqrt{((\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1)^{-1} + \boldsymbol{\Sigma}_2^{-1})^{-1}}) &\geq \frac{1}{\sqrt{2}} \min\{\sigma_{\min}(\boldsymbol{\Sigma}_0 + \boldsymbol{\Sigma}_1), \sigma_{\min}(\boldsymbol{\Sigma}_2)\} \\ &\geq \frac{1}{\sqrt{2}} \min\{\sigma_{\min}(\boldsymbol{\Sigma}_0), \sigma_{\min}(\boldsymbol{\Sigma}_1), \sigma_{\min}(\boldsymbol{\Sigma}_2)\} \\ &\geq \frac{1}{\sqrt{2}} \hat{\sigma} \geq \|\mathbf{B}_2\|_{\text{len}}\eta_\epsilon^+(\mathbb{Z}^n) \end{aligned}$$

holds by the hypothesis $\hat{\sigma} \geq \sqrt{2}\hat{B}\eta_\epsilon^+(\mathbb{Z}^n) \geq \sqrt{2}\|\mathbf{B}_2\|_{\text{len}}\eta_\epsilon^+(\mathbb{Z}^n)$. Hence, we obtain (33) by Cor. 3.9. Repeating the above, we obtain (31). $\square$